

HOSTELHUB - BOARDING HOUSE MANAGEMENT SYSTEM

FINAL GRADUATION PROJECT REPORT
PHAN ANH NGUYỄN - GCH230207

Contents

LIST OF ABBREVIATIONS.....	5
ACKNOWLEDGEMENTS.....	8
ABSTRACT.....	8
CHAPTER 1: INTRODUCTION	10
1.1 Background and Context of Study	10
1.2 Motivation.....	11
1.3 Problem Statement	12
1.4 Objectives.....	13
1.5 Scope of the Project.....	14
1.6 Report Structure	15
CHAPTER 2: LITERATURE REVIEW	16
2.1 Background Knowledge	16
2.1.1 Web Application Architecture.....	16
2.1.2 RESTful API Design	16
2.1.3 Relational Database Management Systems and ACID Guarantees	16
2.1.4 Asynchronous I/O and the Event-Driven Model	17
2.2 Related Technologies and Methodologies	17
2.2.1 Node.js Versus Python with Django.....	17
2.2.2 MySQL Versus MongoDB	18
2.2.3 JWT Versus Session-Based Authentication	18
2.3 Selected Technologies and Methodologies	19
2.4 Related Work and Existing Systems	20
2.4.1 Manual and Spreadsheet-Based Methods.....	20
2.4.2 General-Purpose SaaS Property Management Platforms.....	20
2.4.3 Existing Boarding House Management Applications in Vietnam.....	21
2.5 Scope of HostelHub Compared with Similar Systems.....	21
CHAPTER 3: REQUIREMENT ANALYSIS	24
3.1 Overview of Requirement Analysis Methodology	24

3.2 Functional Requirements	24
3.2.1 Administrator Requirement	25
3.2.2 Tenant Requirements	26
3.3 Non-Functional Requirements	27
3.4 Use Case Diagram Description	28
3.5 Use Case Specifications	33
3.6 Business Rules and Constraints	37
CHAPTER 4: SYSTEM DESIGN	38
4.1 Architectural Overview	38
4.1.1 Presentation Layer	39
4.1.2 Application Layer	40
4.1.3 Data Layer	40
4.2 Database Design and Schema	41
4.3 Security Architecture	47
4.3.1 Password Hashing with Bcrypt	47
4.3.2 JWT Stateless Session Management	47
4.3.3 VNPay HMAC-SHA512 Request Signing and IPN Validation	47
4.4 RESTful API Architecture Design	49
4.5 Application Flow and Sequence Diagrams	52
4.5.1 Contract Creation with Atomic Transaction Handling	52
4.5.2 VNPay Payment Execution and IPN Cryptographic Verification	55
4.6 User Interface and UX Design Principles	57
4.6.1 System Sitemap	57
4.6.2 Low-Fidelity Wireframes	57
4.6.3 Implementation Principles	59
CHAPTER 5: IMPLEMENTATION AND TESTING	60
5.1 Development Environment and Technology Stack	60
5.2 Key Feature Implementation and Core Code Snippets	61
5.2.1 Native Async Error Handling with Express v5	61

5.2.2 Application-Level Timezone Synchronization	62
5.2.3 Atomic Transaction in Contract Creation and Service Registration	64
5.2.4 Cryptographic IPN Verification and VNPay Payment Processing	66
5.3 Technical Challenges and Solutions Summary	68
5.4 Software Testing and Quality Assurance	69
5.4.1 Testing Environment and Seed Dataset	69
5.4.2 Test Case Execution Matrix	69
5.4.3 Test Results and Defect Resolution	72
CHAPTER 6: EVALUATION	78
6.1 Evaluation Objectives and Methodology	78
6.2 System Performance and Stress Testing	78
6.3 Security Compliance and Integrity Assessment.....	80
6.3.1 Role-Based Access Control	80
6.3.2 HMAC-SHA512 Payment Signature Verification	80
6.3.3 Input Sanitization and Injection Prevention	81
6.3.4 Data Residency and Regulatory Alignment.....	81
6.4 Limitations.....	81
6.4.1 Absence of a Native Mobile Application.....	81
6.4.2 Email-Only Notification Channel.....	82
6.4.3 Single-Server Deployment Architecture	82
CHAPTER 7: CONCLUSION AND FUTURE WORK	83
7.1 Conclusion.....	83
7.2 Future Work	84
7.2.1 Native Mobile Application	84
7.2.2 Multi-Channel Notification Integration via Zalo ZNS and SMS	85
7.2.3 Predictive Analytics Using Machine Learning for Utility Forecasting and Revenue Modeling	85
8. REFERENCES	86
APPENDIX: CORE SOURCE CODE IMPLEMENTATION	88
Appendix A: Automated Cron Job Scheduler for Payment Reminders.....	88

A.1 Source Code	88
A.2 Technical Explanation	92
Appendix B: VNPay Secure Payment Gateway Integration	92
B.1 Source Code	92
B.2 Technical Explanation.....	96
Table 1: Comparative Analysis of Hostel Management Solutions	22
Table 2: Core Functional Requirements for System Administrators	25
Table 3: Functional Requirements for Tenants	26
Table 4: System Non-Functional Requirements	27
Table 5: Use Case Specification for Create Rental Contract.....	33
Table 6: Use Case Specification for Enter Utility Meter Readings.....	34
Table 7: Use Case Specification for Generate Monthly Invoices.....	35
Table 8: Use Case Specification for Execute VNPay Payment	36
Table 9: Business Rules and Constraints	37
Table 10: RESTful API Endpoints Specification	49
Table 11: System Technology Stack and Architectural Rationale.....	60
Table 12: Summary of Technical Challenges, Architectural Solutions, and Verification Methods	68
Table 13: Test Case Execution Matrix.....	69
Table 14: System Performance and Stress Testing Results	79
Figure 1: Admin usecase	31
Figure 2: Tenant usecase	32
Figure 3: Three-Tier System Architecture of HostelHub	39
Figure 4: Complete Entity-Relationship Diagram of HostelHub.....	41
Figure 5: Contract Creation & Atomic Transaction	54
Figure 6: VNPay IPN & Signature Verification	56
Figure 7: System sitemap	57
Figure 8: Administrator Dashboard.....	58
Figure 9: Tenant Portal	58
Figure 10: Express v5 centralized JSON error response captured in PowerShell CLI, demonstrating global middleware interception without route-level try-catch handling.....	61

Figure 11: Server startup log in PowerShell CLI confirming application-level timezone alignment (Asia/Ho_Chi_Minh / UTC+07:00) for mysql2 connection pool and scheduled cron tasks..... 62

Figure 12: Execution response of atomic room reservation transaction captured in PowerShell CLI, demonstrating all three table writes committed atomically within one transaction boundary.. 64

Figure 13: Successful execution of VNPay IPN cryptographic HMAC-SHA512 checksum verification in PowerShell CLI, returning response code '00' (Confirm Success) for automated invoice state synchronization..... 66

Figure 14: Administrator Analytics Dashboard 73

Figure 15: Rental Contract Creation Form 74

Figure 16: Utility Meter Entry with Anomaly Warning Dialog 75

Figure 17: Tenant Portal Invoice View and Payment Initiation 76

Figure 18: VNPay Sandbox Payment Execution Interface 77

LIST OF ABBREVIATIONS

ABBREVIATION	FULL TERM
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BR	Business Rule
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
DOM	Document Object Model
ENV	Environment Configuration File
ERD	Entity-Relationship Diagram
ERP	Enterprise Resource Planning
FIPS	Federal Information Processing Standard
FK	Foreign Key

FR	Functional Requirement
GSO	General Statistics Office of Vietnam
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICT	Information and Communications Technology
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
INNODB	InnoDB Storage Engine
IPN	Instant Payment Notification
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
NODE.JS	Node JavaScript Runtime Environment
NPM	Node Package Manager
NOSQL	Not Only SQL
ORM	Object-Relational Mapping
OS	Operating System
OWASP	Open Web Application Security Project
PCI	Payment Card Industry
PK	Primary Key
QA	Quality Assurance
QC	Quality Control

RAM	Random Access Memory
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
REST	Representational State Transfer
RSS	Resident Set Size
SAAS	Software as a Service
SDLC	Software Development Life Cycle
SHA	Secure Hash Algorithm
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
VND	Vietnamese Dong
VNPAY	Vietnam Payment Solutions Joint Stock Company
XAMPP	Cross-Platform Apache, MySQL, PHP, and Perl
XSS	Cross-Site Scripting
ZNS	Zalo Notification Service

ACKNOWLEDGEMENTS

The completion of this Capstone Project represents the culmination of sustained academic study at FPT Greenwich University, and it would not have been possible without the guidance, encouragement, and institutional support of a number of individuals to whom the author wishes to express sincere gratitude.

First and foremost, the author extends profound thanks to Mrs Huong, Lecturer in the Department of Computing at FPT Greenwich University, for providing consistent academic supervision throughout the duration of this project. The supervisor's constructive feedback on system architecture, technical writing, and requirement analysis methodology fundamentally shaped the quality and direction of this report.

The author also thanks the Faculty of Computing at FPT Greenwich University for providing access to laboratory facilities, research resources, and the academic environment in which this project was conceived and developed.

Finally, the author acknowledges the unwavering support of family members throughout the duration of the programme. Their patience, encouragement, and practical support provided the personal foundation upon which this academic work rests.

ABSTRACT

The informal boarding house rental market, known locally as “*nhà trọ*”, constitutes one of the largest segments of the urban housing sector in Vietnam, accommodating an estimated 3.7 million internal migrants concentrated primarily in Hanoi and Ho Chi Minh City. Despite the scale of this market, the majority of boarding house operators continue to manage rental contracts, monthly billing, and tenant communications through manual record-keeping methods or general-purpose spreadsheet applications. These workflows are structurally incapable of automating recurring billing computations, detecting data entry errors in utility meter readings, or integrating with domestic electronic payment infrastructure.

This report documents the design, implementation, and evaluation of HostelHub, a full-stack web-based boarding house management system developed to address these operational limitations. The system is built on a Node.js runtime using the Express version 5 framework, a MySQL 8.0 relational database with InnoDB storage, the VNPay domestic payment gateway integrated through HMAC-SHA512 cryptographic signing, and the node-cron library for automated background task scheduling.

HostelHub satisfies eighteen functional requirements and seven non-functional requirements derived through structured observation and user interviews, covering the complete rental lifecycle from contract creation through invoice generation and online payment. Six key technical features characterise the system's approach to automation, security, and data integrity. First, a utility consumption anomaly detection mechanism flags utility consumption levels that exceed 150% of the prior month's usage, requiring explicit administrator confirmation before the anomalous record is persisted. Second, an atomic transaction pattern using explicit BEGIN, COMMIT, and ROLLBACK statements guarantees database consistency across multi-table contract operations under concurrent load. Third, a server-to-server VNPay IPN callback verification system recomputes the HMAC-SHA512 signature independently on every received callback, rejecting any notification whose digest does not match before any database change is made. Fourth, an automated cron-scheduled payment reminder system executes on the fifth day of each month, delivering simultaneous email and in-application notifications to all tenants with outstanding invoices without administrator intervention. Fifth, a layered role-based access control system enforces administrator and tenant privilege separation through dedicated JWT-verification and role-inspection middleware, applied before any route handler executes. Sixth, the self-hosted deployment model ensures full data sovereignty and data privacy by storing all tenant personal data and financial records exclusively on infrastructure controlled by the system operator, with no tenant data transmitted to external cloud services.

A test execution matrix of twenty test cases, covering authentication, contract management, utility anomaly detection, invoice generation, payment security, and tenant data scoping, was executed with all cases returning a passing result. Performance evaluation confirmed that all API responses remain within the two-second latency ceiling specified by the non-functional requirements under representative concurrent load.

The system demonstrates that a purpose-built, self-hosted web application can deliver integrated billing automation, secure payment processing, and data sovereignty compliance at zero recurring licensing cost for the Vietnamese boarding house operator segment.

Keywords: boarding house management, Node.js, Express, MySQL, VNPay, HMAC-SHA512, RESTful API, rental billing automation

CHAPTER 1: INTRODUCTION

1.1 Background and Context of Study

Vietnam's rental housing market has expanded at a pace that institutional infrastructure simply has not matched. According to the General Statistics Office of Vietnam (GSO, 2023), the country's urban population grew from approximately 30.5% in 2010 to over 38% in 2023, translating to millions of people relocating annually from rural provinces to industrial zones and major cities such as Ho Chi Minh City, Hanoi, Da Nang, and Binh Duong. This migration wave created, and continues to sustain, a structural demand for low-cost rental housing, which is the segment most commonly served by private boarding house operators referred to in Vietnamese as nhà trọ.

The scale of this market is significant. Estimates from the Vietnam Association of Realtors (VARs, 2022) place the number of informal boarding house establishments in the millions nationwide, the vast majority of which are operated by individual landlords managing anywhere from five to several dozen rooms across one or more physical properties. These operators are not property developers. They are ordinary citizens, including retired civil servants, small business owners, and factory workers who saved enough to construct an extra building, and they manage their properties with whatever tools are immediately available to them: paper ledgers, physical receipt books, messaging applications, and at best a shared spreadsheet.

The consequences of this operational model are concrete and recurring. Rent collection relies on the landlord physically visiting each tenant or waiting for cash payment at the end of the month. Utility billing requires electricity and water consumption to be computed manually, a process that begins when the landlord walks to each room, records meter readings in a notebook, subtracts the previous month's figure, and multiplies by the unit price, which may differ from one property to another. A single arithmetic error in this chain cascades: a tenant disputes the figure, the landlord has no audit trail, and the relationship deteriorates. Maintenance requests are communicated verbally or through informal messages, with no systematic tracking of whether a reported issue was ever resolved. Financial reporting, which a responsible landlord needs in order to understand profitability and plan expenditure, is produced through hours of manual reconciliation, when it is produced at all.

This operational reality is not unique to Vietnam. Studies on small-scale property management in Southeast Asia consistently identify the same cluster of problems: lack of transaction transparency, no centralized tenant data, absence of automated billing, and inability to scale operations without proportional increases in administrative labor (Nguyen & Tran, 2021; Lim et al., 2019). The difference in Vietnam is the sheer volume of the market and the speed at which

it is growing, which makes the cost of continued manual management increasingly unsustainable.

Digital property management solutions do exist at the enterprise level. Platforms such as Buildium, AppFolio, and MRI Software serve professional real estate operators in Western markets with comprehensive, cloud-based management suites. In Vietnam, a handful of domestic software vendors, including MISA AMIS and several SaaS startups, offer basic rental management modules. However, these solutions either carry licensing costs that are inaccessible to individual boarding house operators, require technical expertise that the target user base does not possess, or are designed for apartment complexes operating under formal legal contracts rather than the semi-informal, month-to-month arrangements that characterize the nhà trọ market. None of them integrates a Vietnamese-native payment gateway as a first-class feature.

This gap between the operational needs of the market and the available tooling is the precise context in which HostelHub was conceived and built.

1.2 Motivation

The motivation for building HostelHub is direct. It stems from observing real administrative failures, not hypothetical scenarios.

Consider the electricity billing workflow alone. A landlord managing thirty rooms across two properties spends, conservatively, three to four hours per month on meter reading and manual invoice preparation. Errors are common. A transposed digit, for example recording 1,340 kWh instead of the actual 1,034 kWh, produces an invoice that the tenant correctly rejects, though only after the payment due date has passed. The correction requires a new handwritten document. There is no version history. There is no automated sanity check. The landlord absorbs the dispute cost in time and in tenant goodwill.

HostelHub directly addresses this failure mode. The system's utility recording endpoint, implemented specifically in the file routes/utilities.routes.js, computes consumption automatically from submitted meter readings, where electricity usage equals the new reading minus the old reading. It then cross-validates the result against the previous month's figure stored in the Utilities table of the database. If the computed consumption exceeds 150% of the prior month's figure, the API returns a structured warning response with the field is_abnormal set to true and refuses to persist the record unless the operator explicitly confirms the anomaly by supplying force_save set to true in the request body. This is not a cosmetic feature. It is a

deliberate defense against data entry errors that, in a manual workflow, would surface only as billing disputes weeks later.

The second motivating failure is rent collection latency. Landlords in the traditional model often wait two to three weeks for full monthly collection because there is no standardized, low-friction payment mechanism available to their tenants. Cash collection is slow. Bank transfers require the tenant to look up account details and initiate the payment manually. HostelHub integrates VNPay, which is Vietnam's dominant domestic payment network processing over one billion transactions annually as of 2023 (VNPay, 2023), directly into the invoicing workflow. A tenant receives an email containing a ready-made payment link. One click redirects them to VNPay's secure payment interface. Successful payment triggers a server-to-server IPN callback to the route at `/api/payment/vnpay ipn`, which verifies the HMAC-SHA512 cryptographic signature and marks the invoice as paid in the database atomically. The landlord does not need to monitor a bank account or chase individual tenants. The system records the transaction and updates the invoice status without manual intervention.

These two cases, specifically the automated anomaly detection built into utility recording and the integrated online payment flow through VNPay, represent the core of what motivated this project. There is a solvable problem. The technology required to solve it is mature and accessible. The target users are underserved by every existing product currently on the market. HostelHub is the practical, purposefully scoped response to that combination of factors.

1.3 Problem Statement

The problem HostelHub addresses exists in three layers, each building directly on the one before it.

At the operational layer, individual boarding house landlords in Vietnam lack a unified digital tool to manage their core responsibilities. Tenant records exist in notebooks or spreadsheets that are not linked to room assignments. Utility readings are recorded independently of billing. Invoices are handwritten and carry no persistent digital form. Service charges, such as parking fees or internet subscriptions, are tracked separately from monthly rent and are frequently omitted when calculating total dues. Each of these disconnected information sources forces the landlord to perform manual reconciliation every billing cycle, introducing both time cost and error risk in equal measure.

At the financial layer, the absence of an integrated payment mechanism creates collection delays that affect cash flow for the landlord and generate friction for tenants. A tenant who wants to pay on time has no guaranteed, instant method to do so without visiting the landlord

in person or initiating an unverified bank transfer. There is no programmatic confirmation on either side. The landlord cannot automatically confirm receipt of payment. The tenant retains no machine-readable record of having paid.

At the analytical layer, landlords have no real-time visibility into the health of their portfolio. They cannot answer, without manual counting, how many rooms are currently occupied, how many invoices remain outstanding from prior months, or which rental contracts are expiring within the next thirty days. Decisions about property investment or pricing adjustments are therefore made without factual basis.

These three layers describe a problem that general-purpose tools cannot adequately address. Spreadsheets handle data storage but provide no payment integration, no automated billing logic, and no tenant-facing interface. Messaging applications facilitate communication but produce no persistent, structured records. The problem requires a purpose-built system that manages data, automates computation, and connects both landlord and tenant to a shared, consistent information state.

1.4 Objectives

The objective of this project is to design, implement, and evaluate HostelHub, a web-based boarding house management system characterized by the following core capabilities:

- **Role-Based Authentication:** We implement a role-based authentication system that distinguishes between administrators and tenants, issues JSON Web Tokens with a seven-day validity period upon successful login, and enforces role-specific access control across all protected API routes, as implemented in the `auth.routes.js` file.
- **Property Hierarchy Model:** We build a property hierarchy model that organizes rooms under named houses, where each house record stores its own per-unit electricity price and water price. This design, represented by the `Houses` and `Rooms` tables in the database schema, allows a single operator to manage multiple properties with differing utility tariff structures from one account without duplicating configuration.
- **Rental Contract Management:** We implement a rental contract management module that creates formal booking records linking tenants to rooms, captures deposit amounts and contract date ranges, and enforces a closure policy that blocks contract termination if any unpaid invoices remain against the associated room, as defined in the booking management route file.

- **Utility Consumption Monitoring:** We build a utility recording system that accepts monthly meter readings, computes electricity and water consumption automatically by subtracting the old reading from the new reading, and flags utility consumption levels that exceed 150% of the prior month's usage, requiring administrator confirmation before the record is saved.
- **Automated Invoice Generation:** We automate monthly invoice generation by aggregating base rent, computed utility costs at house-specific rates, and applicable service charges, with support for per-month service exclusions that allow administrators to waive individual charges for a specific billing period without modifying the underlying contract terms.
- **Secure Payment Integration:** We integrate VNPay's domestic payment gateway using HMAC-SHA512 request signing, supporting both in-application payment flows and redirect-based payment links delivered by email, with a server-to-server callback handler that atomically updates invoice status in the database upon confirmed payment receipt.

1.5 Scope of the Project

The scope of this project covers the administrative and financial operations of a multi-property boarding house business managed by a single operator or organization.

In scope. The system manages user accounts for both administrators and tenants. It organizes properties into a two-level hierarchy of houses and rooms. It records and tracks rental contracts alongside services registered to each contract. It records monthly utility meter readings, detects consumption anomalies, and generates monthly invoices that consolidate rent, utilities, and services into a single billable figure. It provides a VNPay-integrated payment flow accessible to tenants. It supports in-application notifications, tenant-submitted incident reports, house-level announcements from administrators, and automated daily database backups executed through a scheduled mysqldump process.

Out of scope. The system does not handle multi-currency billing or Vietnamese tax computation. It does not integrate with government land registry databases or produce legally binding lease documents in digital form. It does not support real-time chat between landlord and tenant. It does not include a native mobile application; all access is exclusively through a web browser. It does not implement end-to-end encryption for stored tenant personal data, which represents a known limitation that is addressed in the future work section of this report.

1.6 Report Structure

This report is organized into seven chapters.

Chapter 1 establishes the background of the study, identifies the operational problem being solved, and defines the project objectives and scope as described above.

Chapter 2 reviews the technical literature relevant to this project, covering the technologies selected for implementation, existing boarding house management systems against which HostelHub is compared, and the rationale for the architectural decisions made.

Chapter 3 presents the requirement analysis, including the full list of functional and non-functional requirements, use case diagrams, and detailed use case specifications for each major interaction the system supports.

Chapter 4 documents the system design, covering the database schema and entity-relationship model, the REST API structure, the security architecture, and the interface layout for both the administrator dashboard and the tenant portal.

Chapter 5 describes the implementation process, walking through the key technical decisions made during development, the structure of the codebase across its route files and configuration modules, and the specific challenges encountered and resolved during construction.

Chapter 6 reports the testing process, including the test plan, individual test cases mapped to functional requirements, and the recorded results of each test executed against the running system.

Chapter 7 summarizes the project outcomes, evaluates the completed system against its stated objectives, identifies current limitations, and proposes concrete directions for future development.

CHAPTER 2: LITERATURE REVIEW

2.1 Background Knowledge

2.1.1 Web Application Architecture

Modern web applications follow the client-server architectural model, in which a browser-based client sends HTTP requests to a server that processes those requests and returns structured responses. This separation of concerns, formalized in the layered system constraint described by Fielding (2000), allows the client and server to evolve independently as long as the interface between them remains stable. HostelHub follows this model directly: the browser loads static HTML and JavaScript files from the Express server's public directory, and all subsequent data operations are performed through HTTP requests to a dedicated REST API. The two HTML files, specifically the administrator dashboard file and the tenant portal file, contain no business logic of their own. They delegate every data operation to the API.

2.1.2 RESTful API Design

Representational State Transfer, commonly abbreviated as REST, is an architectural style for distributed hypermedia systems introduced by Roy Fielding in his doctoral dissertation at the University of California, Irvine (Fielding, 2000). A RESTful API exposes resources as addressable URLs, uses standard HTTP verbs to express operations on those resources, and returns stateless responses that contain all the information a client needs to understand the result. Richardson and Ruby (2007) identify six constraints that define REST compliance: client-server separation, statelessness, cacheability, a uniform interface, a layered system, and optional code on demand. HostelHub's API adheres to the first five of these constraints across all fourteen route files. Each resource type, including bookings, invoices, utilities, users, and payments, is addressed by a dedicated URL prefix. Operations on those resources use GET for retrieval, POST for creation, PUT for updates, and DELETE for removal.

2.1.3 Relational Database Management Systems and ACID Guarantees

A relational database management system organizes data into tables of rows and columns, enforces integrity through a predefined schema, and supports structured queries through SQL, a query language whose theoretical foundations trace directly to Codd's relational model (Codd, 1970). The critical property of a relational database for financial applications is ACID compliance. The ACID acronym stands for Atomicity, Consistency, Isolation, and Durability, a set of guarantees first formalized by Haerder and Reuter (1983) and subsequently elaborated in the

systems literature by Gray and Reuter (1992). Atomicity ensures that a group of statements either all succeed or all fail together, leaving no partial results. Consistency ensures that every committed transaction brings the database from one valid state to another, respecting all defined constraints. Isolation ensures that concurrent transactions do not observe each other's intermediate states. Durability ensures that committed transactions survive hardware or software failures.

These guarantees are not abstractions. They have direct consequences for HostelHub's data integrity. The booking creation endpoint opens an explicit database transaction, inserts the booking record, registers the associated services row by row into the ServiceRegistrations table, and updates the room's tenant reference in the Rooms table, all within one atomic block. If any operation in this sequence fails, the entire transaction is rolled back. Without atomicity, a partial failure at the service registration step could leave the system in a state where a room is marked as occupied but no booking record exists, a contradiction that would silently corrupt every invoice generated for that room in subsequent months. MySQL's InnoDB storage engine, which HostelHub uses by default through the mysql2 connection pool, provides full ACID compliance at the transaction level (Oracle Corporation, 2023).

2.1.4 Asynchronous I/O and the Event-Driven Model

JavaScript's single-threaded execution model, combined with an event-driven, non-blocking I/O architecture, allows a Node.js server to handle many concurrent connections without spawning a new operating system thread for each incoming request (Tilkov & Vinoski, 2010). This model suits applications where operations are I/O-bound rather than CPU-bound. A database query yields control back to the event loop while waiting for MySQL to respond, allowing the process to serve other requests in the interim. The Node.js Foundation (2023) identifies this event loop mechanism as the core reason why Node.js achieves high throughput on data-intensive applications with minimal memory overhead per connection. HostelHub's route handlers use the `async/await` syntax throughout, built on top of mysql2's promise-based API, which is the idiomatic application of this non-blocking model in modern Node.js development.

2.2 Related Technologies and Methodologies

2.2.1 Node.js Versus Python with Django

Node.js and Python with the Django framework represent two widely adopted platforms for building web API servers. Django provides a batteries-included philosophy, shipping with an ORM, a built-in admin interface, and authentication middleware as first-party components

(Holovaty & Kaplan-Moss, 2009). This accelerates initial development for teams already proficient in Python. However, Django's synchronous request handling model, while partially mitigated by the ASGI specification in recent versions, introduces thread-per-request overhead under high concurrency that Node.js avoids by design. Node.js also shares a single language with the browser-side JavaScript that powers HostelHub's frontend, which eliminates the context-switching cost of maintaining two separate language environments across the full development team. For a project where the same developers write both the frontend event handlers and the backend route logic, this unification represents a concrete productivity advantage.

2.2.2 MySQL Versus MongoDB

MongoDB is a document-oriented NoSQL database that stores records as JSON-like documents without a predefined schema (Chodorow, 2013). This flexibility accelerates early prototyping when data structures are uncertain or evolving. It comes at a significant cost for financial applications, however. MongoDB's multi-document ACID transactions, available since version 4.0, require explicit session management and carry coordination overhead that is absent from MySQL's native transaction model. Schema flexibility, one of MongoDB's headline features, also increases the risk of inconsistent data representations across documents when multiple application versions write to the same collection. HostelHub's data is inherently relational: a booking references a room, a room references a house, an invoice references a booking and a room, and a payment references an invoice. These relationships are expressed and enforced automatically through foreign key constraints in MySQL, constraints that would require full application-level enforcement in MongoDB. For a system where a corrupted invoice record carries direct financial consequences for the operator and tenant alike, MySQL's schema rigidity is an asset rather than a limitation.

2.2.3 JWT Versus Session-Based Authentication

Session-based authentication stores session state on the server, typically in memory or in a dedicated key-value store such as Redis, and delivers the client a session identifier through a cookie. JSON Web Tokens, specified formally in RFC 7519 by Jones, Bradley, and Sakimura (2015), encode all authentication claims directly into a cryptographically signed token that the client stores locally and presents on every subsequent request. The server verifies the token's signature against a shared secret without consulting any stored state. This stateless design fits HostelHub's single-server architecture because it requires no session store infrastructure and simplifies horizontal scaling if the application grows. The practical trade-off is that individual

tokens cannot be revoked before their expiry without implementing a server-side blacklist, which reintroduces the very state that JWT aims to eliminate. HostelHub accepts this trade-off given its single-deployment context and a seven-day token lifetime, which limits the exposure window of any compromised credential to a manageable period.

2.3 Selected Technologies and Methodologies

The technology choices for HostelHub reflect direct responses to specific technical requirements, not convenience defaults.

Node.js with Express version 5 serves as the runtime and HTTP framework. Express 5 introduces native propagation of errors thrown inside async route handlers, forwarding them automatically to the framework's error middleware. HostelHub wraps every route handler in a utility function defined in the file `utils/asyncHandler.js`, which captures any rejected promise and passes the error to Express's next function, producing consistent error responses across all endpoints without duplicating try-catch logic.

MySQL, accessed through the `mysql2` library's promise-based connection pool, serves as the database engine. The choice is grounded in the ACID requirements described in section 2.1.3. The booking creation route, the booking closure route, and the service removal route all open explicit transaction blocks because each modifies multiple tables whose integrity must be guaranteed atomically, as detailed in the files `routes/bookings.routes.js` and `routes/payment.routes.js`.

JWT authentication, implemented through the `jsonwebtoken` library with a seven-day expiry and a server-held secret key, satisfies the stateless authentication requirement without requiring a session store.

VNPay is selected as the payment gateway because it is the dominant domestic payment network in Vietnam and provides a sandbox environment for development and testing. The HMAC-SHA512 signature scheme that VNPay mandates for all payment URLs and IPN callbacks provides message authentication, ensuring that HostelHub can verify, on every callback received at the IPN route, that the request was genuinely issued by VNPay and has not been modified in transit. HMAC-SHA512 is defined in NIST FIPS PUB 198-1 as a keyed-hash message authentication code that combines a cryptographic hash function with a shared secret key to produce an authentication tag (National Institute of Standards and Technology, 2008). This verification is the cryptographic foundation of the payment security model.

Nodemailer with Gmail SMTP handles outbound email for payment reminders. **node-cron** schedules two time-based recurring tasks: the monthly payment reminder job executing

on the fifth day of each month at 08:00, and the nightly database backup job that calls mysqldump at 02:00. Both are time-triggered rather than event-triggered, making a cron scheduler the correct tool rather than an event queue or a message broker.

2.4 Related Work and Existing Systems

2.4.1 Manual and Spreadsheet-Based Methods

The baseline against which any boarding house management system must be measured is the manual workflow that the majority of Vietnamese landlords currently use. This workflow combines paper-based rent ledgers with consumer spreadsheet applications such as Microsoft Excel or Google Sheets. Research on manual record-keeping in small business contexts consistently identifies three structural failure modes: transcription error accumulation, zero audit trail for disputed figures, and the complete inability to automate recurring computations (Bhimani, 2003; Collier, 2015). In the boarding house context, these failures manifest as arithmetic errors in utility billing, lost payment records, and the inability to generate a financial summary across a multi-property portfolio without hours of manual aggregation.

Spreadsheets address the data storage problem partially. A landlord can record meter readings in a column and write a subtraction formula to compute consumption. However, a spreadsheet has no concept of a tenant record, a rental contract, a service registration, or a payment status. Each of these entities must live in a separate sheet, and the relationships between them must be maintained manually through cell references that break silently when rows are inserted or deleted. There is no mechanism to detect anomalous consumption, send a reminder email, or record a VNPay transaction against a specific invoice. These are not limitations that can be engineered around within a spreadsheet. They require a purpose-built application.

2.4.2 General-Purpose SaaS Property Management Platforms

Enterprise property management platforms such as Buildium, AppFolio, and MRI Software occupy the opposite end of the spectrum. These systems offer comprehensive feature sets covering tenant screening, lease management, maintenance tracking, accounting, and online rent collection (Buildium, 2023; AppFolio, 2023). They are designed for professional property management companies operating large residential or commercial portfolios in Western markets.

Three factors make these platforms unsuitable for the Vietnamese nhà trọ market. First, their subscription pricing, which typically ranges from USD 50 to several hundred dollars per month

depending on unit count, exceeds what an individual landlord managing ten to thirty rooms can justify. Second, they are designed for markets with formal lease agreements and credit-card-based rent collection. None integrates with VNPay or any Vietnamese domestic payment network. Third, the operational complexity of configuring and maintaining an enterprise SaaS platform assumes a level of technical literacy that the target user base does not have. A landlord who currently manages billing through a notebook cannot be expected to configure webhook endpoints or accounting chart-of-accounts mappings.

Domestic Vietnamese alternatives, including rental management modules offered by MISA AMIS, address the language and cultural barrier but share a similar structural constraint: they are components within larger ERP-oriented suites rather than purpose-built boarding house management tools, and their pricing models reflect the small business accounting market rather than the individual landlord segment (MISA, 2023).

2.4.3 Existing Boarding House Management Applications in Vietnam

A number of lightweight mobile applications targeting the Vietnamese boarding house market are available on the Google Play Store and iOS App Store under names such as Quản Lý Nhà Trọ and similar variants. These applications typically support room listing, basic tenant contact storage, and manual payment recording. Several offer a monthly fee calculator where the landlord inputs electricity and water readings to compute a bill.

What these applications consistently lack is system integration. The billing computation runs in isolation from a tenant account, so the tenant receives no notification and has no portal to view or dispute the bill. Payment is recorded by the landlord after collection, with no connection to any payment network. There is no automated reminder mechanism, no contract lifecycle management, and no administrative analytics view. These applications function as digital notebooks, which is an improvement over paper, but they do not constitute a management system in any integrated sense.

2.5 Scope of HostelHub Compared with Similar Systems

The following table places HostelHub against the three categories of existing solutions across eight evaluative criteria that reflect the core requirements identified in Chapter 3 of this report.

Table 1: Comparative Analysis of Hostel Management Solutions

CRITERION	MANUAL / SPREADSHEET	ENTERPRISE SAAS	LOCAL MOBILE APPS	HOSTELHUB
LICENSING COST	Near zero	USD 50– 500/month	Free to low- cost	Zero (self-hosted)
SETUP COMPLEXITY	None	High	Low	Moderate
AUTOMATED INVOICE GENERATION	None	Full	Partial (calculator only)	Full (rent + utilities + services per booking period)
VNPAY PAYMENT INTEGRATION	None	None	None	Native (HMAC- SHA512, IPN callback)
TENANT-FACING PORTAL	None	Full	None	Full (invoices, payment, incident reporting)
UTILITY ANOMALY DETECTION	None	None	None	Yes (flags consumption exceeding 150% of prior month)
AUTOMATED PAYMENT REMINDERS	None	Full	None	Full (scheduled email and in-app notification)
DATA OWNERSHIP AND CONTROL	Full (local)	Vendor- controlled	Varies	Full (self-hosted MySQL)

The table makes HostelHub's positioning clear. It does not compete with enterprise SaaS on feature breadth, and it does not attempt to do so. It does not offer tenant credit screening, multi-currency accounting, or legal document generation. What it provides is the precise subset of functionality that a Vietnamese boarding house operator needs, integrated as a coherent system rather than a collection of disconnected tools, and at zero recurring licensing cost.

Two differentiating capabilities stand out because no existing solution in any category provides them together. The first is native VNPay integration covering the full payment lifecycle: invoice generation, signed payment URL construction using HMAC-SHA512, server-to-server IPN callback verification, and atomic invoice status update in the database. The second is the utility consumption anomaly detection, which operates automatically on every meter reading submission and flags values exceeding 150% of the prior month's recorded figure without requiring any administrator configuration. Both capabilities are implemented at the server level as first-class features, not as optional add-ons.

Data control represents a third meaningful differentiator. Enterprise SaaS platforms store all tenant and financial data on vendor-controlled cloud infrastructure. This raises data residency concerns that are directly relevant under Vietnam's Cybersecurity Law, which took effect in 2019 and imposes obligations on organizations storing Vietnamese citizen data to do so on domestically located servers (National Assembly of Vietnam, 2018). HostelHub operates on a self-hosted MySQL database that the operator controls entirely. No tenant data leaves the operator's server. This architecture satisfies the residency requirement by default, without any additional configuration or legal agreement with a cloud vendor.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Overview of Requirement Analysis Methodology

Requirements for HostelHub were gathered through a combination of direct observation of existing boarding house management workflows and structured interviews with two target user groups: boarding house operators who currently manage properties using manual or spreadsheet-based methods, and tenants who currently pay rent through cash collection or unverified bank transfers.

The observation phase focused on documenting the end-to-end workflow of a single billing cycle, from meter reading to payment confirmation. Observers recorded the sequence of steps a landlord performs, identified the points where errors most frequently occur, and noted the information artifacts produced at each stage. This process revealed four critical workflow breakdowns: utility billing without automated verification, invoice delivery without a formal notification channel, payment collection without a programmatic confirmation mechanism, and maintenance tracking without a structured request system.

The interview phase used open-ended questions to gather requirements from the tenant's perspective. Tenants were asked how they currently receive billing information, what methods they use to pay, how they report maintenance problems, and what improvements they would value most. The responses consistently prioritized two capabilities: receiving a clear, itemized bill in digital form rather than verbally, and having a payment method that does not require a physical visit or a manual bank transfer.

The gathered observations and interview responses were translated into formal requirements using a methodology aligned with IEEE Standard 830, which specifies that each requirement must be stated as a verifiable, unambiguous condition on externally observable system behavior (IEEE, 1998). Requirements are assigned unique identifiers for traceability and are divided by user role.

3.2 Functional Requirements

Functional requirements describe what the system must do as observable behaviors from the user's perspective. They are organized into two groups based on the role of the actor performing each action.

3.2.1 Administrator Requirement

Table 2: Core Functional Requirements for System Administrators

ID	REQUIREMENT	DESCRIPTION
FR-01	User Account Management	The system must allow an administrator to create, view, update, and deactivate user accounts for both tenants and other administrators. Deactivated accounts must be blocked from logging in without being permanently deleted.
FR-02	Property Hierarchy Management	The system must allow an administrator to create and manage boarding house records, each assigned its own address and per-unit utility pricing. Rooms must be created within a specific house and assigned a base rent value and floor area.
FR-03	Rental Contract Creation and Closure	The system must allow an administrator to create a rental contract linking one tenant to one room, specifying a start date, end date, base rent, and deposit amount. The system must prevent contract creation for any room that already has an active tenant. It must also prevent contract closure while any unpaid invoice exists for the associated room.
FR-04	Service Definition and Registration	The system must allow an administrator to define available services with a name, unit price, and charge type. Services must be assignable to specific contracts, and individual service charges must be waivable for a specific billing month without modifying the underlying contract terms.
FR-05	Utility Meter Reading Entry	The system must allow an administrator to enter monthly electricity and water meter readings for each room, providing both the previous and current meter values. The system must calculate consumption automatically. It must warn the administrator and require explicit confirmation if recorded consumption exceeds 150% of the prior month's figure for the same room.
FR-06	Monthly Invoice Generation	The system must allow an administrator to trigger invoice generation for a selected month and year. Each generated invoice must aggregate base rent, computed utility costs calculated at the rate assigned to the room's parent house, and applicable service charges for every active contract covering that billing period.

FR-07	Invoice Status Management	The system must allow an administrator to manually mark an invoice as paid and to delete invoices where required.
FR-08	Payment Reminder Dispatch	The system must allow an administrator to send a payment reminder for a specific unpaid invoice. The reminder must be delivered as an email containing a direct payment link and simultaneously as an in-application notification visible in the tenant's portal.
FR-09	Incident and Maintenance Management	The system must allow an administrator to view all maintenance and incident reports submitted by tenants and to update the status of each report across three states: pending, in progress, and resolved.
FR-10	Announcement Publishing	The system must allow an administrator to publish announcements targeted either to all tenants within a specific house or to all tenants across all managed properties.
FR-11	Administrative Analytics Dashboard	The system must display a summary view showing current room occupancy rates, total and monthly revenue from paid invoices, the count of pending maintenance incidents, a list of invoices that remain unpaid from prior billing months, and a list of rental contracts expiring within thirty days.
FR-12	Automated Database Backup	The system must automatically create a full backup of the database once per day at a scheduled time, storing the resulting file in a designated local directory accessible to the server operator.

3.2.2 Tenant Requirements

Table 3: Functional Requirements for Tenants

ID	REQUIREMENT	DESCRIPTION
FR-13	Account Registration and Login	The system must allow a new user to self-register as a tenant by providing a full name, email address, phone number, and password. Registered tenants must be able to log in using their phone number and password combination.
FR-14	Invoice Viewing	The system must display to a logged-in tenant all invoices associated with their assigned room, showing the billing period,

		itemized charges for base rent, utilities, and services, the total amount due, and the current payment status. The system must prevent any tenant from viewing invoices associated with a different tenant's room.
FR-15	Online Payment via VNPay	The system must allow a tenant to initiate payment for an unpaid invoice through VNPay's payment interface. Upon payment confirmation from VNPay, the invoice status must be updated to paid automatically, without requiring any action from the administrator.
FR-16	Incident Reporting	The system must allow a tenant to submit a maintenance or incident report describing a problem in their room, including a title and a written description. Tenants must be able to view the current status of all reports they have previously submitted.
FR-17	Notifications and Announcements	The system must display in-application notifications sent either by the system or by the administrator and must show all announcements published for the tenant's assigned house. Tenants must be able to mark individual notifications as read.
FR-18	Profile Management	The system must allow a tenant to update their name and email address. A password change must require the tenant to provide their current password before the new password is accepted.

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints the system must satisfy, independent of any specific feature.

Table 4: System Non-Functional Requirements

ID	CATEGORY	REQUIREMENT
NFR-01	Performance	All API responses for standard read and write operations must complete within 500 milliseconds under normal conditions with up to fifty concurrent active users. Invoice generation across all active contracts for a single billing month must complete within ten seconds regardless of the number of rooms processed.

NFR-02	Security	All user passwords must be stored as cryptographic hashes and must never be stored or transmitted in plain text at any point in the system. All endpoints except login and self-registration must require a valid authentication token. Payment callbacks received from VNPay must be verified against a cryptographic signature before any database modification is performed.
NFR-03	Availability	The system must remain operational without planned downtime during standard business hours. Automated backup processes must execute in the background without interrupting or degrading the experience of active users.
NFR-04	Usability	The system must be accessible through any modern web browser without requiring the installation of additional software on the user's device. Both the administrator interface and the tenant portal must be navigable by users with no specialized technical training.
NFR-05	Data Integrity	Operations that modify multiple records as a single logical unit, including contract creation, contract closure, and service removal from a contract, must execute atomically. A failure at any step within such an operation must leave the system in its state prior to the operation, with no partial changes persisted.
NFR-06	Regulatory Compliance	The system must store all tenant personal data and financial records on infrastructure controlled directly by the operator, consistent with the data localization obligations specified in Vietnam's Cybersecurity Law enacted in 2018 (National Assembly of Vietnam, 2018).
NFR-07	Maintainability	The system must be organized by functional domain, such that a developer can locate, understand, and modify the logic for any specific feature without altering or risking unrelated components of the system.

3.4 Use Case Diagram Description

HostelHub supports two primary actors who interact with the system in fundamentally distinct ways.

The **Administrator** actor represents the boarding house operator or property manager holding full system access. The Administrator can perform all management operations, including user account management, property and room configuration, contract lifecycle management, service definition, utility data entry, invoice generation, payment reminder dispatch, and financial analytics review.

The **Tenant** actor represents a registered occupant whose room is covered by an active rental contract. The Tenant's interactions with the system are scoped strictly to their own data: viewing invoices, initiating payments, submitting incident reports, reading announcements, and managing their own profile.

The use cases associated with each actor are as follows.

Administrator Use Cases:

- UC-A01: Manage User Accounts
- UC-A02: Manage Houses and Rooms
- UC-A03: Create Rental Contract
- UC-A04: Close Rental Contract
- UC-A05: Define and Assign Services
- UC-A06: Enter Utility Meter Readings
- UC-A07: Generate Monthly Invoices
- UC-A08: Manually Update Invoice Status
- UC-A09: Send Payment Reminder
- UC-A10: Manage Incident Reports
- UC-A11: Publish House Announcements
- UC-A12: View Analytics Dashboard
- UC-A13: Trigger Database Backup

Tenant Use Cases:

- UC-T01: Self-Register Account
- UC-T02: Log In
- UC-T03: View Own Invoices

- UC-T04: Execute VNPay Payment
- UC-T05: Submit Incident Report
- UC-T06: View Notifications and Announcements
- UC-T07: Update Personal Profile

Both actors share the authentication entry point, though the system routes them to separate interfaces upon login based on their assigned role. The Administrator and Tenant roles are mutually exclusive per account; no single account may hold both roles simultaneously.

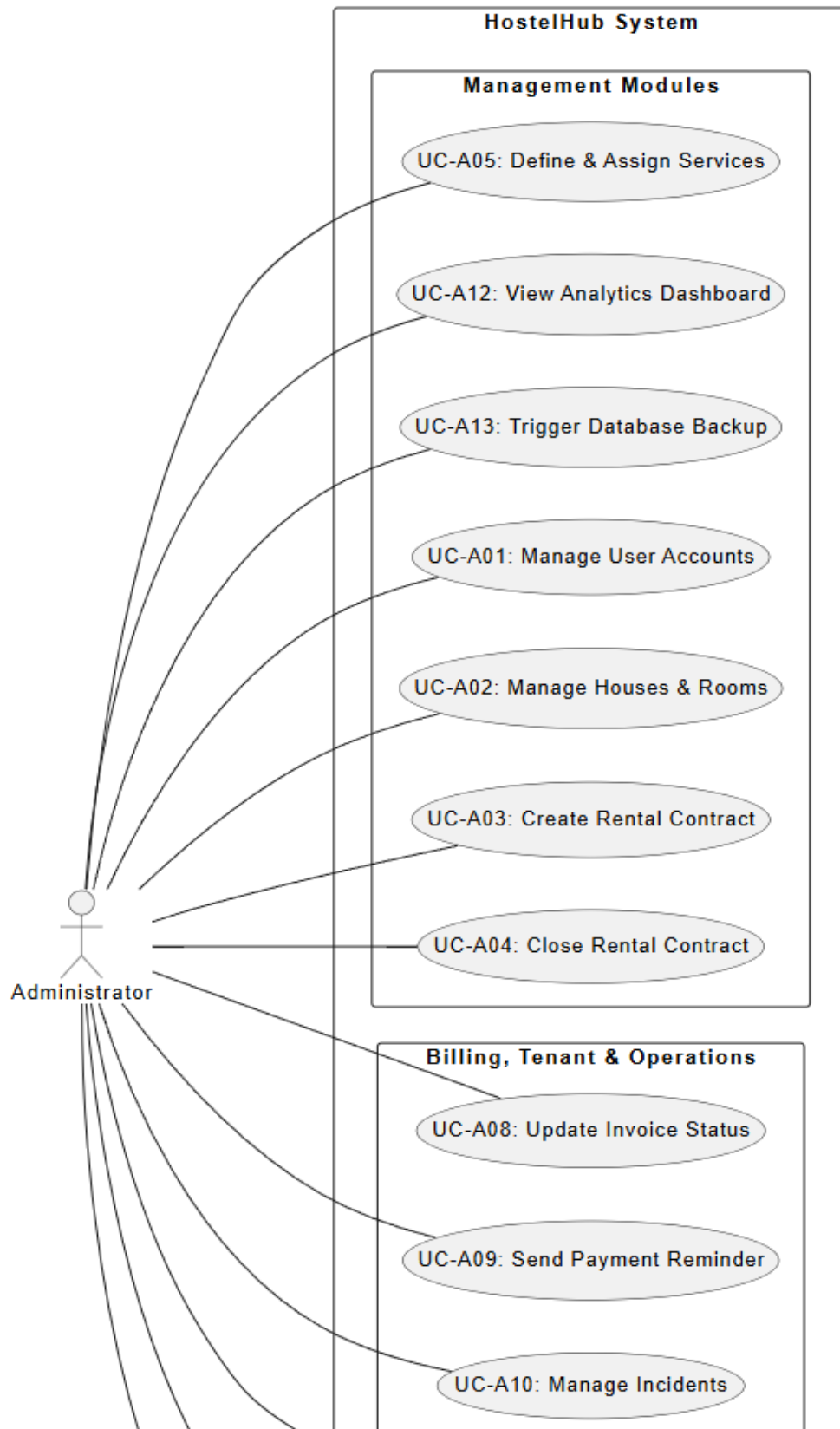


Figure 1: Admin usecase

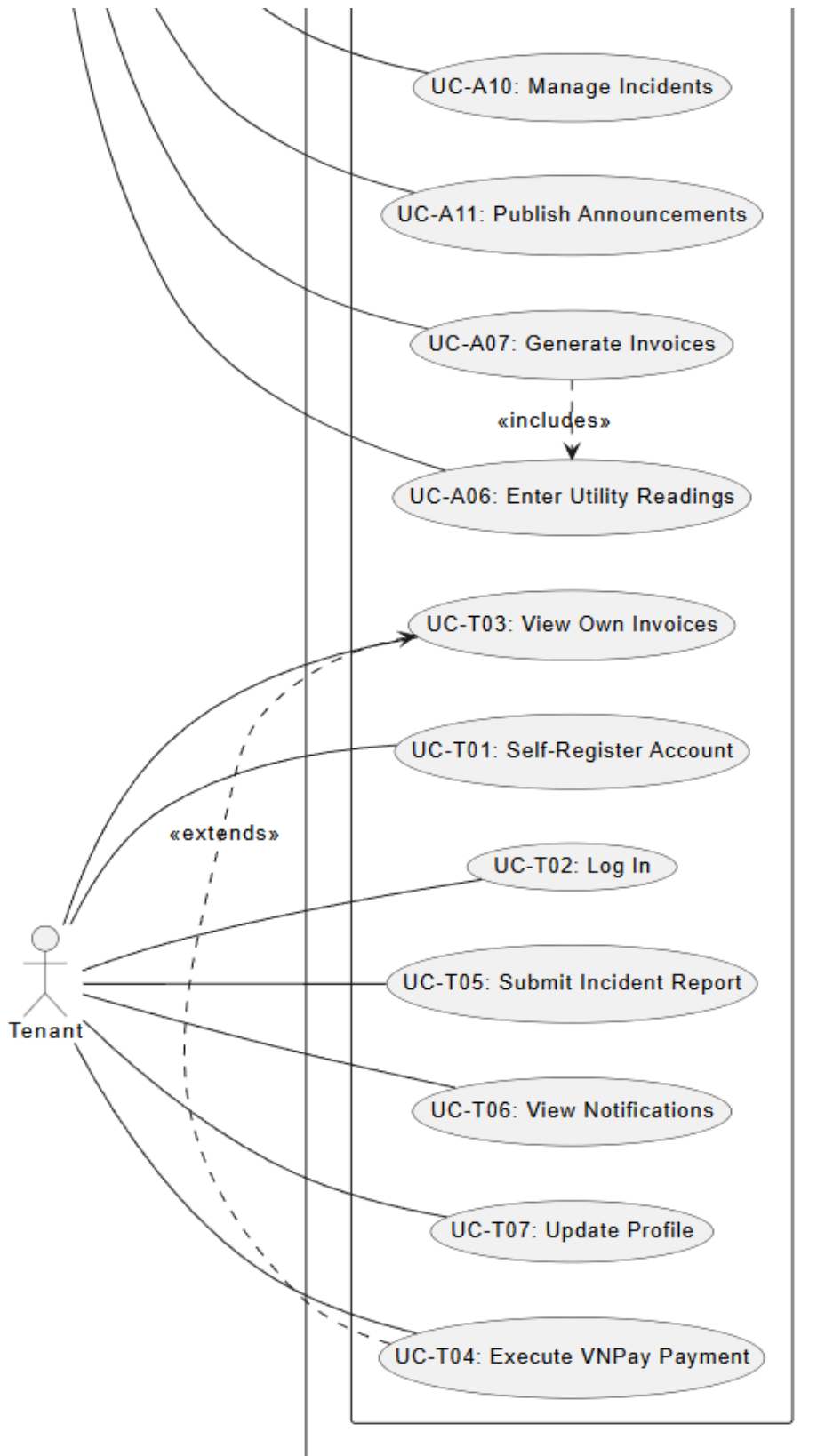


Figure 2: Tenant usecase

3.5 Use Case Specifications

UC-01: Contract Creation and Validation

Table 5: Use Case Specification for Create Rental Contract

FIELD	DETAIL
USE CASE ID	UC-A03
USE CASE NAME	Create Rental Contract
ACTOR	Administrator
PRECONDITIONS	The administrator is authenticated. The target tenant account exists and is in active status. The target room exists within a house and currently has no active tenant assigned to it.
POSTCONDITIONS	A new rental contract record links the selected tenant to the selected room. The room is marked as occupied. All selected services are registered against the new contract.

Main Flow:

1. The administrator navigates to the contract creation screen.
2. The administrator selects a registered tenant from the user list.
3. The administrator selects a room from the list of rooms that currently have no active occupant.
4. The administrator provides a contract start date, end date, agreed base rent, and deposit amount.
5. The administrator optionally selects one or more available services to register against the contract.
6. The administrator submits the form.
7. The system verifies that the selected room has no current active tenant.
8. The system creates the contract record, registers each selected service, and updates the room to reflect the new occupant.
9. The system returns a confirmation message.

Alternative Flow A — Room Occupied at Submission: At step 7, if another tenant has been assigned to the room between the administrator opening the form and submitting it, the system

rejects the request and displays a conflict notification. No contract record is created and the room's state is unchanged.

Exception Flow — Partial Failure During Creation: If any step in the creation sequence fails after it has begun, the system reverses all changes made up to that point. No contract record, service registration, or room occupancy change is persisted.

UC-02: Utility Entry with Anomaly Detection

Table 6: Use Case Specification for Enter Utility Meter Readings

FIELD	DETAIL
USE CASE ID	UC-A06
USE CASE NAME	Enter Utility Meter Readings
ACTOR	Administrator
PRECONDITIONS	The administrator is authenticated. The target room has an active rental contract for the billing month being recorded.
POSTCONDITIONS	A utility record for the specified room and billing period exists in the system. If consumption was confirmed despite exceeding the anomaly threshold, the record carries an anomaly flag.

Main Flow:

1. The administrator selects a room and a billing month and year.
2. The administrator enters the previous and current electricity meter values.
3. The administrator enters the previous and current water meter values.
4. The administrator submits the readings.
5. The system calculates electricity and water consumption from the submitted values.
6. The system retrieves the consumption figures recorded for the same room in the immediately preceding month.
7. Consumption for both utilities falls within 150% of the prior month's figure. The system saves the utility record and returns a success response.

Alternative Flow A — Anomaly Detected: At step 7, if calculated consumption for either utility exceeds 150% of the prior month's figure, the system returns a warning response identifying the

anomaly along with the computed consumption values. The record is not saved. The administrator reviews the figures and, if they are confirmed as accurate, resubmits the readings with an explicit confirmation flag. The system saves the record and marks it as anomalous.

Exception Flow — Current Reading Below Previous Reading: At step 5, if the current meter value for either utility is lower than the previous value submitted, the system immediately rejects the request and returns an error. No record is created or modified.

UC-03: Monthly Invoice Generation

Table 7: Use Case Specification for Generate Monthly Invoices

FIELD	DETAIL
USE CASE ID	UC-A07
USE CASE NAME	Generate Monthly Invoices
ACTOR	Administrator
PRECONDITIONS	The administrator is authenticated. At least one rental contract is active during the selected billing month.
POSTCONDITIONS	Invoice records exist for all active contracts covering the selected billing period. All newly created invoices carry a status of unpaid.

Main Flow:

1. The administrator selects a target billing month and year and initiates invoice generation.
2. The system identifies all rental contracts whose date range overlaps with the selected billing month.
3. For each identified contract, the system retrieves the agreed base rent, the utility consumption figures recorded for the billing period, and the list of services registered to the contract.
4. The system checks whether any registered services have been explicitly excluded for this specific month.
5. The system calculates the total invoice amount by summing the base rent, utility costs computed at the per-unit rate assigned to the room's parent house, and the charges for all non-excluded services.

6. If no invoice exists for this contract and billing period, the system creates a new unpaid invoice record.
7. If an unpaid invoice already exists for this contract and billing period, the system updates its line items to reflect the current calculation.
8. The system reports the count of invoices created and invoices updated.

Exception Flow — No Active Contracts in Period: If no contracts are active during the selected billing month, the system returns a message indicating that invoice generation cannot proceed and takes no further action.

UC-04: VNPay Payment Execution

Table 8: Use Case Specification for Execute VNPay Payment

FIELD	DETAIL
USE CASE ID	UC-T04
USE CASE NAME	Execute VNPay Payment
ACTOR	Tenant
PRECONDITIONS	The tenant is authenticated. At least one invoice associated with the tenant's room carries a status of unpaid.
POSTCONDITIONS	The invoice status is updated to paid. A payment transaction record is created, storing the payment amount and the VNPay transaction identifier linked to the invoice.

Main Flow:

1. The tenant navigates to their invoice list and selects an unpaid invoice.
2. The tenant initiates the payment action.
3. The system constructs a signed payment request and redirects the tenant's browser to VNPay's hosted payment interface.
4. The tenant completes the payment on VNPay's platform using their preferred method.
5. VNPay sends a server-to-server confirmation to the system with the transaction result.
6. The system verifies the authenticity of the confirmation using a cryptographic signature check against the shared secret.

7. The signature is valid and the transaction result is confirmed as successful. The system updates the invoice to paid status and stores the transaction record.
8. VNPay redirects the tenant's browser to the portal, which now displays the invoice with a paid status.

Alternative Flow A — Payment Cancelled or Declined: If the tenant cancels the payment or VNPay returns a non-success response code, the browser is redirected to the portal with an error message. The invoice remains unpaid. No payment record is created.

Exception Flow — Signature Verification Failure: At step 6, if the cryptographic signature in the VNPay confirmation does not match the value the system computes independently, the system rejects the notification entirely and makes no database change. The invoice status remains unpaid.

3.6 Business Rules and Constraints

The following business rules apply system-wide and enforce the operational policies of the boarding house management domain.

Table 9: Business Rules and Constraints

ID	BUSINESS RULE	RATIONALE
BR-01	A rental contract cannot be closed while any unpaid invoice exists for the associated room.	Ensures all financial obligations are settled before a tenant vacates, preventing unresolved debt from persisting after the contract is archived.
BR-02	Utility consumption exceeding 150% of the prior month's recorded figure triggers a mandatory confirmation step before the reading is saved.	Protects against data entry errors that would propagate directly into invoice calculations, where a single transposed digit in a meter reading produces an incorrect bill for the tenant.
BR-03	Service charges registered against a contract apply to every billing month within the contract period by default. An administrator may exclude a specific service	Allows temporary waivers for services not consumed in a given month without disrupting the contract's service registration for all other months.

	from a specific month without removing the service from the contract itself.	
BR-04	A tenant account may only access invoices, notifications, and incident reports associated with their own assigned room. Requests for records belonging to other tenants are denied regardless of how the request is constructed.	Enforces data privacy between tenants sharing the same property, ensuring that one tenant cannot access another's financial information.
BR-05	Only one active rental contract may be associated with a given room at any point in time. A new contract cannot be created for a room that already has an active occupant.	Maintains property data integrity and prevents billing conflicts that would arise if two tenants were simultaneously billed for the same room.
BR-06	Invoice generation includes only contracts whose date range overlaps with the selected billing month. Contracts that begin after the billing month ends or conclude before it begins are excluded from that generation run.	Ensures tenants are billed only for months in which their contract was active, respecting agreed contract dates.
BR-07	A payment confirmation received from VNPay must pass cryptographic signature verification before the corresponding invoice status is updated. A confirmation that fails verification is rejected without producing any database change.	Prevents fraudulent or malformed payment notifications from marking invoices as paid, ensuring that only genuine VNPay transactions result in a status update.

CHAPTER 4: SYSTEM DESIGN

4.1 Architectural Overview

HostelHub is organized around a three-tier client-server architecture that separates the system into three distinct layers of responsibility, each with a well-defined interface to the layers adjacent to it.

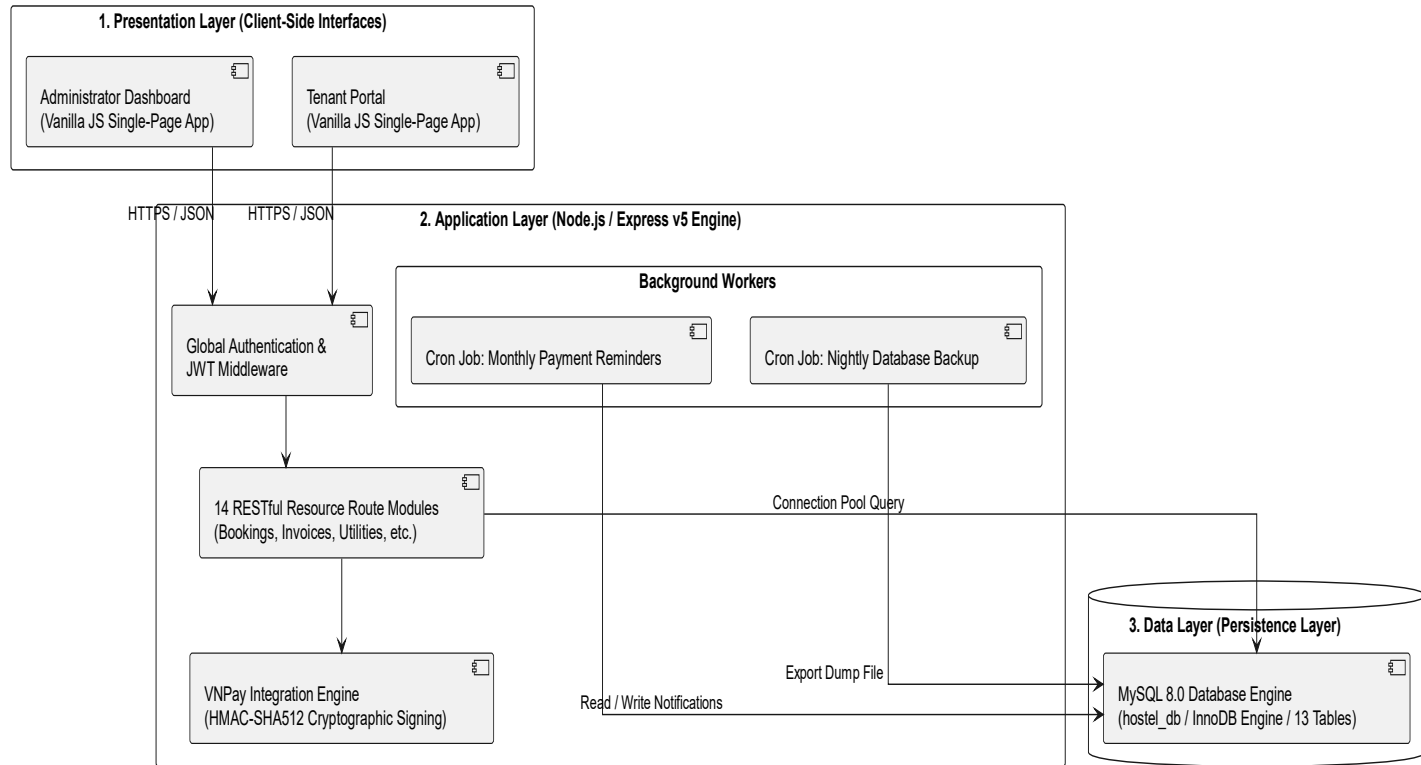


Figure 3: Three-Tier System Architecture of HostelHub

4.1.1 Presentation Layer

The Presentation Layer comprises two static HTML pages served from the server's public directory: the administrator dashboard and the tenant portal. Neither page contains business logic. Both operate as single-page applications that use vanilla JavaScript to make asynchronous HTTP requests to the Application Layer API, parse the JSON responses, and update the document object model accordingly. This design means the server does not render HTML on each request, reducing server-side computation and allowing both interfaces to be cached by the browser. The administrator dashboard JavaScript file exceeds 80 kilobytes, reflecting the breadth of management operations it coordinates. The tenant portal JavaScript file is a more focused 20-kilobyte file covering invoice viewing, payment initiation, incident reporting, and notification management.

4.1.2 Application Layer

The Application Layer is implemented in Node.js using the Express version 5 framework. It receives HTTP requests from the Presentation Layer, enforces authentication and authorization, executes business logic, interacts with the Data Layer through a managed connection pool, and returns JSON-formatted responses. The layer is organized into fourteen route modules, each responsible for a single resource domain such as bookings, invoices, utilities, or payments. A global middleware function intercepts all requests arriving at the API path prefix, validates the token in the Authorization header, and attaches the decoded user identity to the request object before passing control to the appropriate route handler. Two background processes run within this layer: a cron-scheduled email reminder job that executes on the fifth day of each month at 08:00, and a nightly database backup job that invokes `mysqldump` at 02:00.

4.1.3 Data Layer

The Data Layer is a MySQL database named `hostel_db`, configured to use the InnoDB storage engine. InnoDB provides row-level locking, enforced foreign key constraints, and full ACID transaction support, all of which are required by HostelHub's booking creation and payment recording workflows. The database schema consists of thirteen tables. The schema initializes automatically on server startup through a self-executing function that creates missing tables, adds missing columns to existing tables where required, and removes deprecated structures, allowing the schema to evolve without manual database administration between deployments.

4.2 Database Design and Schema

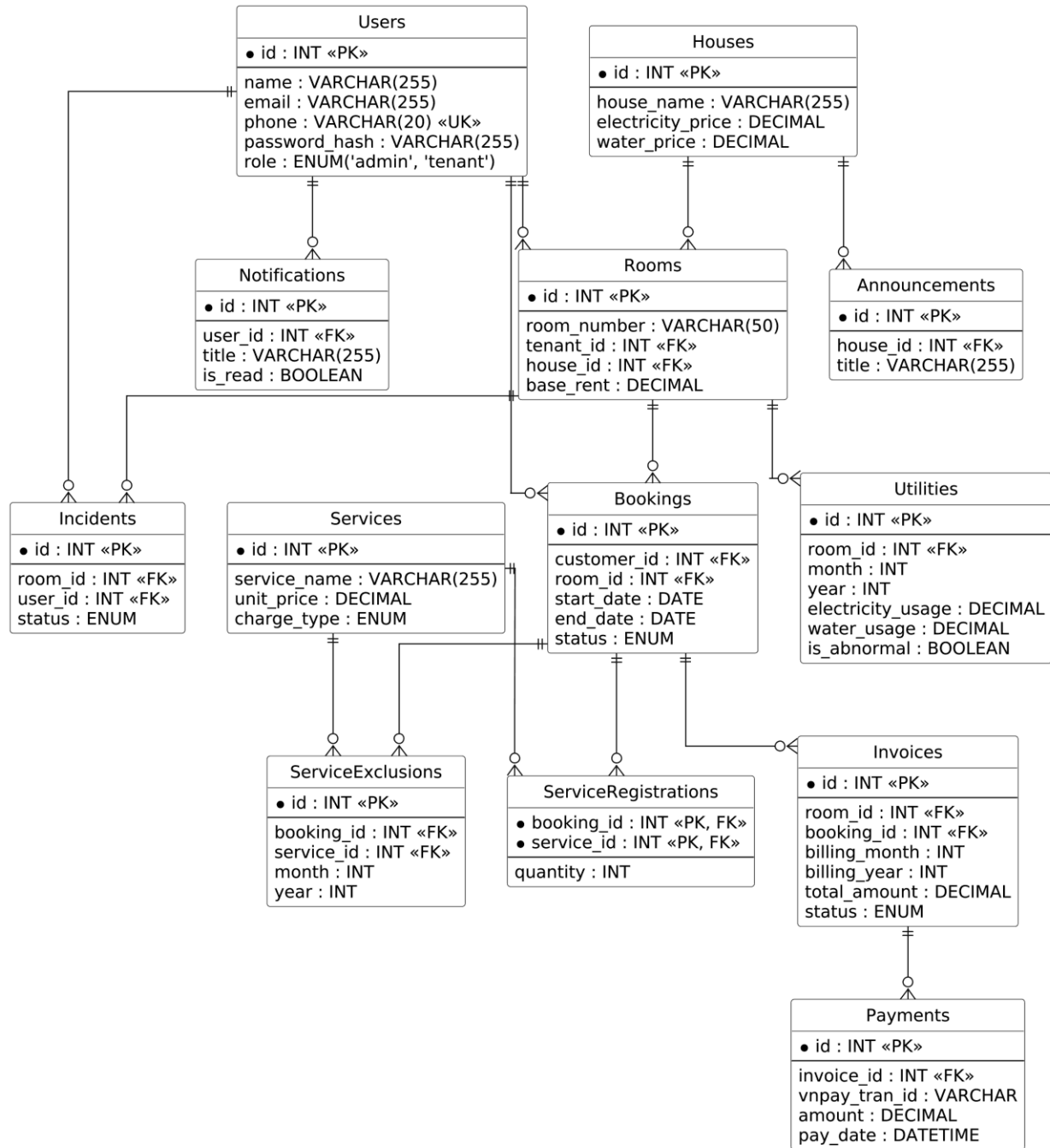


Figure 4: Complete Entity-Relationship Diagram of HostelHub

The following subsections present the complete schema of `hostel_db`. For each table, the field name, data type, and applied constraints are listed. Foreign key relationships and the rationale for key design decisions are noted after each table.

"Note on Terminology: In the database schema, the Bookings table serves as the technical underlying model for Rental Contracts, and the customer_id field represents the tenant entity within the system architecture."

Users

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
NAME	VARCHAR(255)	NOT NULL
EMAIL	VARCHAR(255)	NOT NULL
PHONE	VARCHAR(20)	UNIQUE, NULL
PASSWORD_HASH	VARCHAR(255)	NULL
STATUS	ENUM('active', 'inactive')	DEFAULT 'active'
ROLE	ENUM('admin', 'tenant')	DEFAULT 'tenant'

The phone field serves as the login credential rather than email, reflecting the convention in the Vietnamese rental market where landlords and tenants identify each other by phone number. The password_hash field is nullable to support accounts created by an administrator on behalf of a tenant who has not yet set a password through self-registration.

Houses

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
HOUSE_NAME	VARCHAR(255)	NOT NULL
ADDRESS	VARCHAR(255)	NULL
ELECTRICITY_PRICE	DECIMAL(10,0)	DEFAULT 3500
WATER_PRICE	DECIMAL(10,0)	DEFAULT 20000

Per-unit utility pricing is stored at the house level rather than globally. This design allows a landlord managing multiple properties in different locations, which may carry different utility tariffs set by the local building owner, to configure each property independently without affecting billing for other properties.

Rooms

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
ROOM_NUMBER	VARCHAR(50)	NOT NULL
TENANT_ID	INT	FK → Users(id) ON DELETE SET NULL
BASE_RENT	DECIMAL(15,2)	NOT NULL
HOUSE_ID	INT	FK → Houses(id) ON DELETE SET NULL
AREA	DECIMAL(10,2)	NULL

The tenant_id column records the current occupant as a denormalized reference, enabling fast occupancy lookups without a join to the Bookings table. When a contract is closed, this field is set to NULL, returning the room to available status. The relationship between Houses and Rooms is one-to-many: one house record may be referenced by many room records through the house_id foreign key.

Bookings

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
CUSTOMER_ID	INT	FK → Users(id) ON DELETE CASCADE
ROOM_ID	INT	FK → Rooms(id) ON DELETE SET NULL
START_DATE	DATE	NOT NULL
END_DATE	DATE	NOT NULL
BASE_RENT	DECIMAL(15,2)	NOT NULL
DEPOSIT_AMOUNT	DECIMAL(15,2)	DEFAULT 0.00
STATUS	ENUM('active', 'closed')	DEFAULT 'active'
CREATED_AT	DATETIME	DEFAULT CURRENT_TIMESTAMP

The Bookings table is the contractual backbone of the system. Every invoice and service charge traces to a booking record through the booking_id foreign key. The base_rent column on the booking captures the rate agreed at contract signing independently of the room's current listed price, ensuring historical accuracy when the room's base_rent is updated for future tenants.

Services and ServiceRegistrations

FIELD	TYPE	CONSTRAINTS
SERVICES		
ID	INT	PRIMARY KEY, AUTO_INCREMENT
SERVICE_NAME	VARCHAR(255)	NOT NULL
UNIT_PRICE	DECIMAL(15,2)	NOT NULL
CHARGE_TYPE	ENUM('per_room', 'per_person')	DEFAULT 'per_room'
SERVICEREGISTRATIONS		
BOOKING_ID	INT	COMPOSITE PRIMARY KEY, FK → Bookings(id) ON DELETE CASCADE
SERVICE_ID	INT	COMPOSITE PRIMARY KEY, FK → Services(id) ON DELETE CASCADE
QUANTITY	INT	DEFAULT 1

ServiceRegistrations is a junction table that implements the many-to-many relationship between Bookings and Services. Its composite primary key prevents a duplicate service registration on the same booking.

ServiceExclusions

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
BOOKING_ID	INT	NOT NULL, FK → Bookings(id) ON DELETE CASCADE
SERVICE_ID	INT	NOT NULL, FK → Services(id) ON DELETE CASCADE
MONTH	INT	NOT NULL
YEAR	INT	NOT NULL

Each row in ServiceExclusions records that a specific service registered to a specific booking should not be charged for the given billing month and year. The invoice generation process queries this table and skips the charge for any matching combination, implementing per-month waivers without modifying the underlying service registration.

Utilities

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
ROOM_ID	INT	FK → Rooms(id) ON DELETE SET NULL
MONTH	INT	NOT NULL
YEAR	INT	NOT NULL
OLD_ELECTRICITY	DECIMAL(10,2)	NULL
NEW_ELECTRICITY	DECIMAL(10,2)	NULL
ELECTRICITY_USAGE	DECIMAL(10,2)	NOT NULL
OLD_WATER	DECIMAL(10,2)	NULL
NEW_WATER	DECIMAL(10,2)	NULL
WATER_USAGE	DECIMAL(10,2)	NOT NULL
IS_ABNORMAL	BOOLEAN	DEFAULT FALSE

Both the raw meter readings and the computed consumption values are stored. Storing raw readings allows the administrator to audit the source data and recompute consumption if readings are corrected. The is_abnormal flag persists the anomaly status for reporting purposes even after the administrator confirms and saves the record.

Invoices

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
ROOM_ID	INT	FK → Rooms(id) ON DELETE SET NULL
BOOKING_ID	INT	FK → Bookings(id) ON DELETE SET NULL
BILLING_MONTH	INT	NOT NULL
BILLING_YEAR	INT	NOT NULL
BASE_RENT	DECIMAL(15,2)	NOT NULL

UTILITY_COST	DECIMAL(15,2)	NOT NULL
SERVICE_COST	DECIMAL(15,2)	DEFAULT 0.00
TOTAL_AMOUNT	DECIMAL(15,2)	NOT NULL
STATUS	ENUM('unpaid', 'paid')	DEFAULT 'unpaid'
CREATED_AT	DATETIME	DEFAULT CURRENT_TIMESTAMP

The invoice stores computed line item amounts at generation time. If utility prices or service rates change after an invoice is generated, the stored invoice values remain unchanged, preserving an accurate record of costs at the time of billing.

Payments

FIELD	TYPE	CONSTRAINTS
ID	INT	PRIMARY KEY, AUTO_INCREMENT
INVOICE_ID	INT	NOT NULL, FK → Invoices(id) ON DELETE CASCADE
VNPAY_TRAN_ID	VARCHAR(255)	NOT NULL
AMOUNT	DECIMAL(15,2)	NOT NULL
PAY_DATE	DATETIME	DEFAULT CURRENT_TIMESTAMP

Notifications, Incidents, and Announcements

TABLE	KEY FIELDS	PURPOSE
NOTIFICATIONS	user_id FK, title, message, is_read BOOLEAN, created_at	In-application alerts delivered to individual users
INCIDENTS	room_id FK, user_id FK, title, description, status ENUM('pending', 'in_progress', 'resolved'), created_at	Maintenance and fault reports submitted by tenants
ANNOUNCEMENTS	house_id FK nullable, title, content, created_at	Broadcast messages from administrators scoped to one house or all properties

A null value in `Announcements.house_id` indicates a global announcement visible to all tenants. A non-null value scopes the announcement to tenants whose room belongs to the specified house.

4.3 Security Architecture

4.3.1 Password Hashing with Bcrypt

User passwords are never stored in plain text. Upon account creation or password update, the system passes the submitted password through the bcrypt algorithm with a work factor of 10, producing a 60-character hash stored in the `password_hash` column of the `Users` table. Bcrypt incorporates a randomly generated salt into every hash computation, which ensures that two identical passwords produce different hash outputs, defeating precomputed rainbow table attacks (Provos & Mazières, 1999). At login time, the system recomputes the hash from the submitted credential and compares it to the stored value using a constant-time comparison function, which prevents timing-based side-channel attacks where an attacker could infer partial password matches from response latency differences.

4.3.2 JWT Stateless Session Management

Upon successful authentication, the server generates a JSON Web Token signed with HMAC-SHA256 using a secret key held in the application's environment configuration. The token payload carries the user's database identifier, full name, assigned role, and email address. It is issued with a seven-day expiration claim. The token is returned to the client in the login response body. On every subsequent API request, the client presents the token in the `Authorization` header using the Bearer scheme. The authentication middleware defined in the `auth.routes.js` file extracts the token, verifies the HMAC-SHA256 signature against the server secret, and checks the expiration claim. A request carrying an invalid signature or an expired token is rejected with a 403 status response before it reaches any route handler. No session state is stored on the server. This stateless design adds zero database overhead per authenticated request.

4.3.3 VNPay HMAC-SHA512 Request Signing and IPN Validation

All outgoing VNPay payment requests and all incoming IPN callbacks are authenticated using HMAC-SHA512, which applies the SHA-512 hash function to a concatenated parameter string combined with a shared merchant secret key, as defined in NIST FIPS PUB 198-1 (National

Institute of Standards and Technology, 2008). When constructing a payment URL, the system sorts all request parameters alphabetically by key name, serializes them into a query string, and computes the HMAC-SHA512 digest using the merchant secret. The resulting hash is appended to the URL as the `vnp_SecureHash` parameter.

When VNPay sends an IPN callback to the server confirming a completed transaction, the Application Layer applies a three-step verification process before making any database change.

Step 1: Checksum Verification. Both the `vnp_SecureHash` field and the `vnp_SecureHashType` field are removed from the received parameter set before any further processing. Removing both fields is mandatory because including either in the signed string would produce a different digest from the one VNPay computed, causing a spurious mismatch. The remaining parameters are sorted in the same alphabetical order used during URL construction, serialized with identical encoding rules, and an HMAC-SHA512 digest is independently computed using the shared merchant secret. If the computed digest does not match the extracted `vnp_SecureHash` value, the callback is rejected immediately with `RspCode '97'` and no database modification is made. This step ensures no external actor can forge a payment notification to fraudulently mark an invoice as settled.

Step 2: Order Existence and Amount Verification. After the signature is confirmed valid, the handler retrieves the invoice record from the database using the invoice identifier extracted from `vnp_TxnRef`. It verifies that the record exists and that the `vnp_Amount` value received from VNPay, after dividing by 100 to convert from VNPay's smallest-unit representation, matches the `total_amount` stored in the database. This step prevents parameter tampering attacks where an attacker intercepts a legitimate callback and reduces the `vnp_Amount` before forwarding it, which would corrupt the payment record without triggering a signature failure if the tampered value happened to re-hash correctly.

Step 3: Order Status Check. The handler verifies that the invoice's current status is 'unpaid' before executing the UPDATE. This guard prevents duplicate processing when VNPay retransmits the IPN after a network timeout. If the status is already 'paid', the handler returns `RspCode '00'` to suppress further retries without inserting a duplicate Payments record.

Only the IPN handler is authorized to write database state. The Return URL endpoint, which receives the browser redirect after VNPay processes the payment, performs the same signature verification to display the correct result screen to the tenant, but it never writes to the Invoices or Payments tables. This strict separation ensures that a race condition cannot arise between a browser redirect and the server-to-server callback arriving simultaneously.

4.4 RESTful API Architecture Design

HostelHub's API is built around REST principles as described by Fielding (2000). Resources are exposed as addressable URL paths organized under the /api prefix. Each resource domain occupies its own path segment, and operations on individual resources are identified by numeric ID parameters. HTTP status codes are applied with semantic intent: 200 for successful reads and updates, 201 for successful record creation, 400 for validation failures, 401 for missing authentication tokens, 403 for authorization failures, and 404 for resources that cannot be located. All responses share a consistent JSON envelope structure with a boolean success field and either a data payload or an error message field.

Authentication levels across the API are divided into three tiers. The first tier covers public routes that accept requests without a token: specifically the login, self-registration, and VNPay callback endpoints. The second tier covers routes accessible to any authenticated user, where the response content is filtered by the user's role. The third tier covers routes restricted exclusively to administrators, enforced through a dedicated authorization middleware that verifies the role claim embedded in the decoded JWT payload.

The following table presents the core API endpoints across all functional modules.

Table 10: RESTful API Endpoints Specification

METHOD	PATH	AUTH LEVEL	DESCRIPTION
POST	/api/auth/register-admin	Public	Create a new administrator account
POST	/api/auth/register-tenant	Public	Tenant self-registration with phone, email, and password
POST	/api/auth/login	Public	Authenticate with phone and password; return signed JWT
GET	/api/auth/me	Authenticated	Return the identity record of the current token holder
GET	/api/users	Admin	List all user accounts, excluding password hashes

POST	/api/users	Admin	Create a tenant or admin account on behalf of another user
PUT	/api/users/profile/update	Authenticated	Update own name, email, or password
PUT	/api/users/:id	Admin	Update any user's details, role, or active status
DELETE	/api/users/:id	Admin	Delete a user with no active rental contracts
GET	/api/houses	Authenticated	List all house records with utility pricing
POST	/api/houses	Admin	Create a new house with address and utility rates
PUT	/api/houses/:id	Admin	Update house name, address, or per-unit utility prices
GET	/api/rooms	Authenticated	List all rooms with occupancy status and house reference
POST	/api/rooms	Admin	Create a new room within a specified house
GET	/api/bookings	Admin	List all contracts with tenant, room, and house details
POST	/api/bookings	Admin	Create a contract with atomic service registration
PUT	/api/bookings/:id/close	Admin	Close a contract and vacate the room if no unpaid invoices remain
POST	/api/bookings/:id/add-service	Admin	Register an additional service to an active contract

POST	/api/bookings/:id/service-exclusions	Admin	Toggle a per-month service charge waiver for a contract
GET	/api/utilities/latest/:room_id	Authenticated	Retrieve the most recent meter values recorded for a room
POST	/api/utilities	Admin	Submit monthly meter readings with automatic anomaly detection
GET	/api/invoices	Authenticated	List invoices; administrators receive all, tenants receive only their own
POST	/api/invoices/generate	Admin	Generate invoices for all active contracts in a billing month
GET	/api/invoices/analytics/summary	Admin	Return dashboard statistics: occupancy, revenue, overdue, expiring contracts
GET	/api/invoices/:id/detail	Authenticated	Return full itemized invoice breakdown for display or print
PUT	/api/invoices/:id	Admin	Manually update invoice payment status
POST	/api/payment/create_url	Authenticated	Construct and return a signed VNPay payment URL
GET	/api/payment/vnpay_ipn	Public	Server-to-server IPN callback; verifies signature and records payment
GET	/api/payment/vnpay_return	Public	Browser redirect return after VNPay payment; updates status and redirects tenant

POST	/api/payment/send-reminder/:id	Admin	Send a payment reminder email and in-app notification for an invoice
POST	/api/incidents	Authenticated	Submit a new maintenance or fault report for a room
PUT	/api/incidents/:id	Admin	Update the resolution status of an incident report
POST	/api/announcements	Admin	Publish an announcement scoped to a house or all tenants
GET	/api/backup/trigger	Admin	Manually trigger a full database backup outside the scheduled window

One notable design decision applies to GET /api/invoices. Rather than maintaining two separate endpoints for administrators and tenants, a single route inspects the role claim from the decoded JWT payload and applies a tenant filter at the SQL query level when the requester holds a tenant role. This preserves a uniform URL interface while implementing strict access scoping without duplicating route handler logic.

4.5 Application Flow and Sequence Diagrams

4.5.1 Contract Creation with Atomic Transaction Handling

The contract creation flow is the most operationally sensitive write path in the system, because it modifies three distinct data points in sequence and must guarantee that all three changes persist together or none persist at all.

The sequence begins when the administrator submits the contract form through the browser. The Presentation Layer sends a POST request to /api/bookings, carrying the tenant identifier, room identifier, contract dates, base rent, deposit amount, and the list of services to register. The Application Layer passes the request through the authentication middleware and the admin authorization middleware in sequence. Both must pass before the route handler executes.

The route handler acquires a dedicated connection from the mysql2 pool and opens a transaction using BEGIN. The first operation inside the transaction queries the Rooms record for

the current `tenant_id` value of the selected room. This check must occur inside the transaction boundary. Performing it before the transaction opens would create a race window where a concurrent contract creation request could assign a different tenant to the same room between the check and the subsequent insert. If the queried `tenant_id` is not null, the handler calls `ROLLBACK` immediately and returns a 400 response. No data is modified.

If the room is unoccupied, the handler inserts the new record into the `Bookings` table and captures the auto-generated booking identifier from the insert result. It then iterates through the submitted service list, inserting one row into `ServiceRegistrations` per service using the new booking identifier as the foreign key. The final operation updates `tenant_id` on the `Rooms` record to the new tenant's identifier. The handler calls `COMMIT` only after all three operations succeed without error. Any exception thrown during the sequence triggers the catch block, which calls `ROLLBACK` and returns a 500 response to the client. The connection is released back to the pool in the finally block regardless of whether the transaction committed or rolled back.

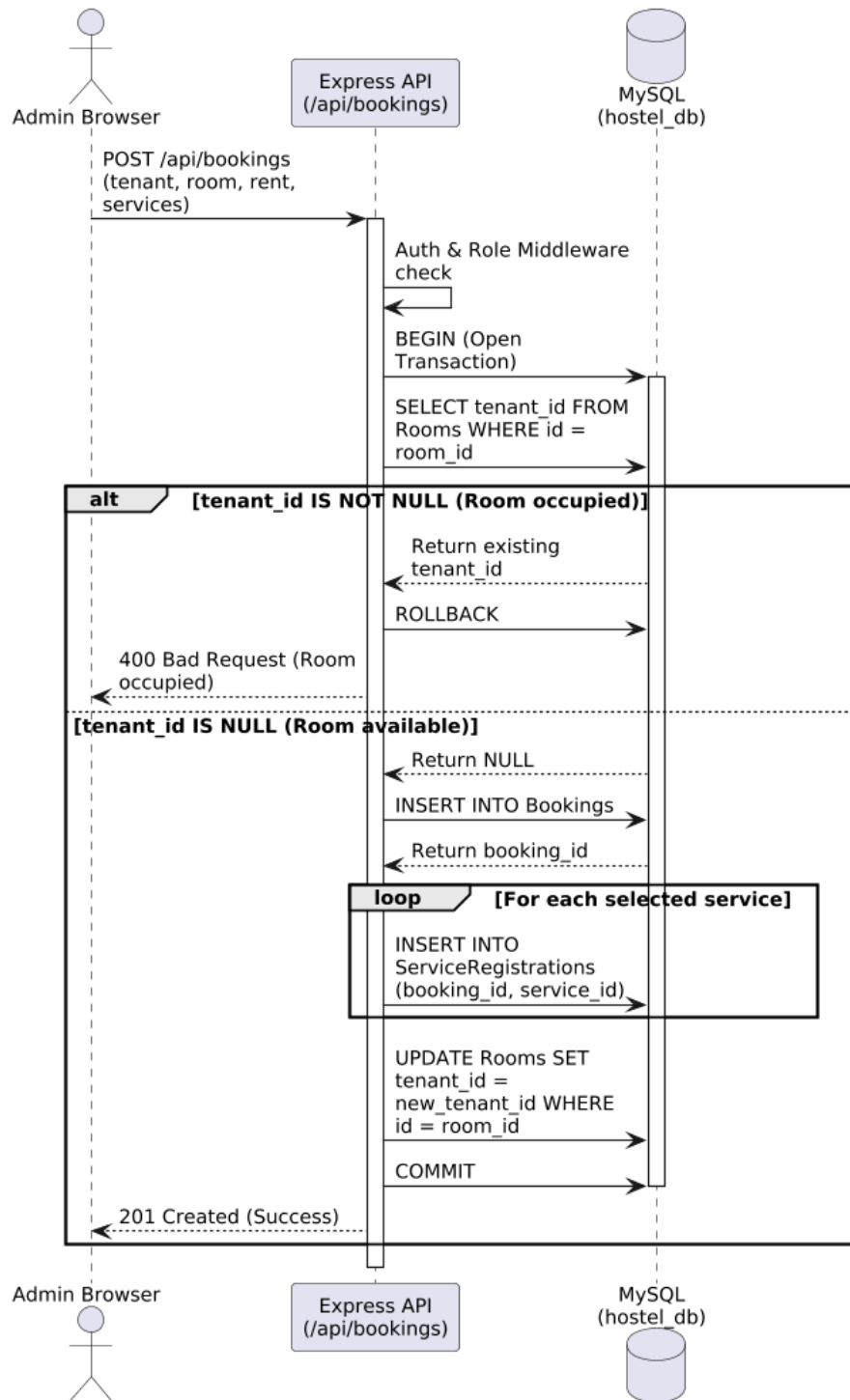


Figure 5: Contract Creation & Atomic Transaction

4.5.2 VNPay Payment Execution and IPN Cryptographic Verification

The VNPay payment flow involves three parties: the tenant's browser, the HostelHub Application Layer, and VNPay's external processing servers. The flow divides into two independent sub-flows that both complete before an invoice is confirmed as paid.

In the first sub-flow, the tenant selects an unpaid invoice in the portal and initiates payment. The browser sends a POST request to `/api/payment/create_url` with the invoice identifier and the total amount. The Application Layer extracts the client IP address from the request headers, assembles the full parameter set required by VNPay's API specification, sorts the parameters alphabetically by key name, serializes them into a URL-encoded string, and computes an HMAC-SHA512 digest using the merchant secret key stored in the environment configuration. The resulting signed URL is returned to the browser, which redirects the tenant to VNPay's hosted payment interface. The tenant selects a bank and completes the payment.

In the second sub-flow, VNPay's servers send a GET request to `/api/payment/vnpay_ipn` containing the full transaction result and a pre-computed HMAC-SHA512 signature in the `vnp_SecureHash` parameter. The Application Layer applies a three-step verification process before making any database change. In Step 1, both the `vnp_SecureHash` field and the `vnp_SecureHashType` field are removed from the received parameter set. Removing both fields is mandatory; including either in the re-signed string would produce a digest that differs from VNPay's computation and cause a spurious mismatch. The remaining parameters are sorted in the same alphabetical order used during URL construction and re-serialized with identical encoding rules. The HMAC-SHA512 digest is independently computed using the shared merchant secret and compared against the extracted `vnp_SecureHash` value. If the values do not match, the handler returns `RspCode '97'` and terminates without touching the database. In Step 2, after the signature is confirmed valid, the handler queries the database using the invoice identifier extracted from `vnp_TxnRef`, verifies that the invoice record exists, and confirms that the `vnp_Amount` value, after dividing by 100 to reverse VNPay's smallest-unit representation, matches the `total_amount` stored in the database. This check prevents parameter tampering attacks where an intercepted callback carries a modified amount that still produces a valid-looking signature. In Step 3, the handler verifies that the invoice's current status is 'unpaid' before executing the UPDATE. This guard prevents duplicate processing when VNPay retransmits the IPN on network timeout. If all three steps pass and `vnp_ResponseCode` equals "00", the handler sets the invoice status to 'paid' and inserts a record into the Payments table with the VNPay transaction identifier and the confirmed amount.

Only the IPN handler is authorized to write database state. The Return URL endpoint, which receives the browser redirect after the tenant completes payment on VNPay's interface, performs the same signature verification to determine which result screen to render for the

tenant, but it never writes to the Invoices or Payments tables. This strict boundary ensures that a race condition cannot arise between a concurrent browser redirect and the server-to-server callback.

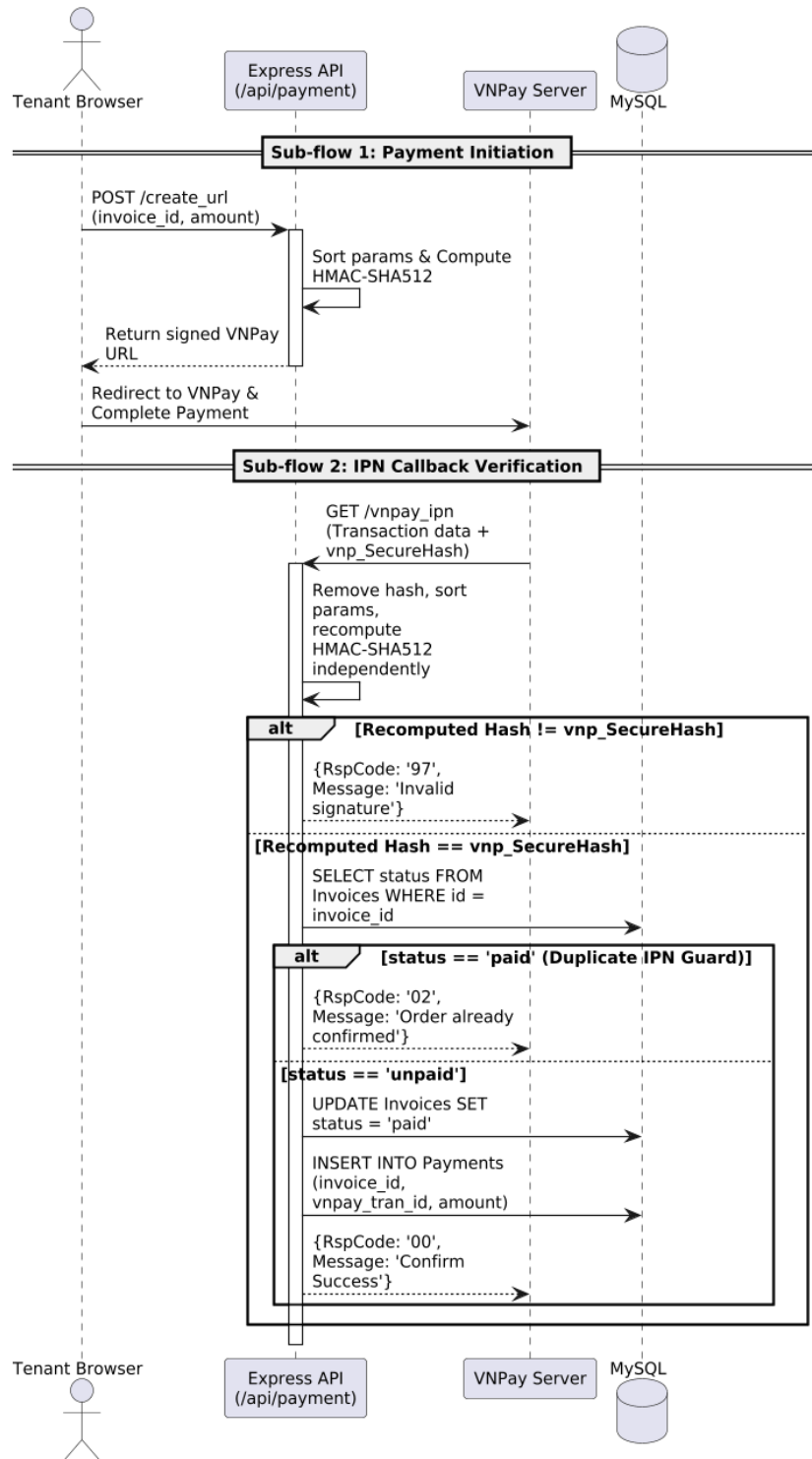


Figure 6: VNPay IPN & Signature Verification

4.6 User Interface and UX Design Principles

4.6.1 System Sitemap

Figure 7 presents the sitemap of HostelHub, illustrating the navigational hierarchy across both user roles. The administrator interface is organized into a single-page application with eight primary module tabs: Dashboard Analytics, Houses and Rooms, Users, Contracts, Utility Entry, Invoices, Services, and Incidents. The tenant interface exposes four sections: My Invoices, Notifications, Incident Reports, and Profile. Public-facing routes, comprising the login page and the VNPay return URL, sit outside both authenticated zones.

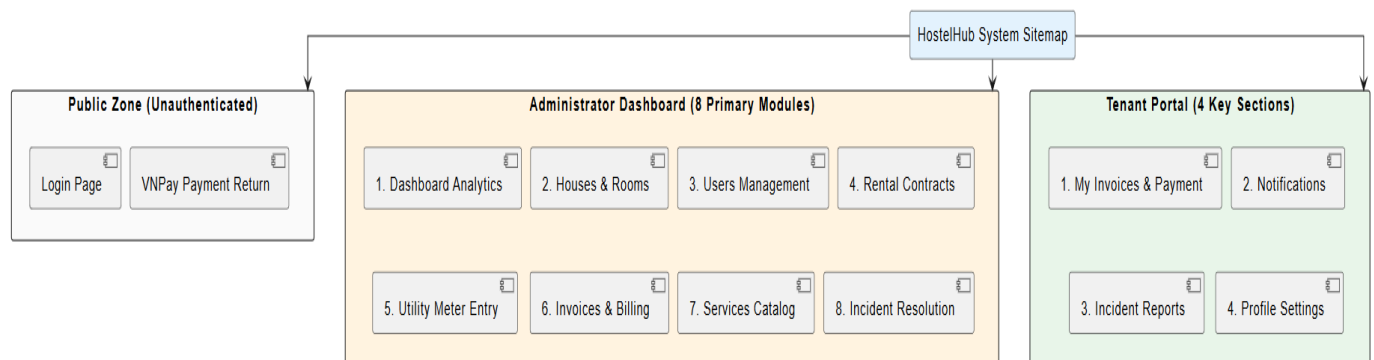


Figure 7: System sitemap

4.6.2 Low-Fidelity Wireframes

Figure 8 and Figure 9 present low-fidelity wireframes produced during the design phase for the two primary interfaces. The administrator wireframe establishes the two-column layout pattern, with a fixed left navigation sidebar listing all module tabs and a main content area occupying the remaining width. The tenant portal wireframe establishes a single-column card layout where each invoice card presents billing period, itemized amounts, and a payment action button. These wireframes served as the structural specification from which the final CSS layout and HTML component hierarchy were derived.

HostelHub Management System - Administrator Portal

Logged in as: Admin

Logout

NAVIGATION MENU

1. Dashboard Analytics

2. Houses & Rooms

3. Users Management

4. Rental Contracts

5. Utility Meter Entry

6. Invoices & Billing

7. Services Catalog

8. Incident Resolution

SYSTEM STATUS

Server Status: Online

DB Backup: Auto Daily

DASHBOARD ANALYTICS OVERVIEW

Filter Month:

October 2025

Apply Filter

Total Revenue: \$12,450.00

Overdue Invoices: 3 Invoices

Occupancy Rate: 92.5% (37/40)

Expiring Contracts: 2

QUICK ACTIONS & SYSTEM ALERTS

+ Generate Invoices

+ Register Contract

+ Submit Readings

RECENT INVOICE ACTIVITY TABLE

Invoice ID	Room	Tenant Name	Amount	Status
INV-2025-0101	Room 101	Nguyen Van A	3,500,000 VND	Paid (VNPay)
INV-2025-0102	Room 102	Tran Thi B	4,100,000 VND	Unpaid

Figure 8: Administrator Dashboard

HostelHub - Tenant Self-Service Portal

Room 101 - Nguyen Van A

Logout

MY INVOICES & PAYMENTS

INVOICE #INV-2025-10 (October 2025)

Status: UNPAID

Due Date: 2025-10-10

Room Rent (Base): 3,000,000 VND

Electricity Usage (120 kWh @ 3,500): 420,000 VND

Water Usage (10 m3 @ 15,000): 150,000 VND

Trash & Service Fee: 50,000 VND

TOTAL AMOUNT DUE: 3,620,000 VND

Pay Now with VNPay (Secure HMAC-SHA512)

INVOICE #INV-2025-09 (September 2025)

Status: PAID

Paid Date: 2025-09-05 (VNPay IPN Confirmed)

TOTAL AMOUNT: 3,500,000 VND

Download PDF Receipt

QUICK TENANT SERVICES

Report New Incident

View Announcements

Edit Profile

Figure 9: Tenant Portal

4.6.3 Implementation Principles

HostelHub's frontend is implemented as two separate single-page applications built without a JavaScript framework, using HTML5 semantic elements, CSS3 for visual presentation, and vanilla JavaScript for all dynamic behavior. This technology choice reflects a deliberate rationale.

Framework-based frontends introduce dependency trees, build pipelines, and bundle overhead that are disproportionate to the complexity of a two-role management application serving a small concurrent user base. By working directly with the browser's native Fetch API, DOM API, and event model, the frontend avoids these overheads entirely while remaining deployable as static files served directly by Express.

The administrator dashboard is the heavier of the two interfaces. Its JavaScript file exceeds 80 kilobytes of unminified code, a size that reflects the breadth of the administrative function set rather than implementation inefficiency. All management modules, covering user management, room and house configuration, contract lifecycle management, utility data entry, invoice generation, payment operations, and analytics, load simultaneously as a single file. Navigation between modules is handled through a tab-based pattern that hides and reveals content sections in the DOM without reloading the page. This eliminates the latency of a full page request for administrators who switch frequently between tasks during a billing cycle.

The tenant portal is intentionally constrained to approximately 20 kilobytes of JavaScript. The portal surfaces only the features a tenant needs: invoice listing with inline payment initiation, incident report submission, notification reading, and profile management. The reduced scope produces a faster initial load, a simpler interaction model, and a clearer visual hierarchy appropriate for users who access the portal once per month to review and pay a bill.

Both interfaces share a common interaction pattern. Form submissions use the Fetch API with JSON request bodies and Authorization headers. API responses are handled through promise resolution functions that update the relevant DOM section on success or display an inline error message on failure. No response, successful or otherwise, triggers a full page reload. This pattern ensures that network errors or validation rejections do not destroy the user's current input or navigation state, which is a meaningful usability consideration for administrators entering multi-field utility readings or contract data.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Development Environment and Technology Stack

The following table presents the complete set of technologies, libraries, and development tools employed in HostelHub, together with their version numbers and the rationale for their inclusion.

Table 11: System Technology Stack and Architectural Rationale

COMPONENT	VERSION	ROLE AND RATIONALE
NODE.JS	20 LTS	Server-side JavaScript runtime providing the event-driven, non-blocking I/O model required for concurrent request handling without thread-per-request overhead
EXPRESS	5.2.1	HTTP routing framework; version 5 was selected specifically for its native async promise-rejection propagation, which eliminates the need for per-route try-catch blocks
MYSQL2	3.22.4	MySQL client library providing a promise-based API and a managed connection pool with configurable concurrency limits
MYSQL	8.0 (InnoDB)	Relational database engine; InnoDB storage provides row-level locking, enforced foreign key constraints, and full ACID transaction support required for atomic contract creation
VANILLA JS	ES2022 (native)	Frontend scripting without a framework; eliminates build pipeline overhead and produces deployable static files served directly by Express
BCRYPTJS	3.0.3	Password hashing library implementing the bcrypt adaptive algorithm with a configurable work factor
JSONWEBTOKEN	9.0.3	JWT signing and verification for stateless, server-side session management
NODE-CRON	4.2.1	Cron-style task scheduler for automated monthly payment reminders and nightly database backups
NODEMAILER	8.0.10	SMTP email transport for reminder dispatch via Gmail relay

DOTENV	17.4.2	Environment variable injection from the .env configuration file at process startup
VNPAY API	v2.1.0	Vietnamese domestic payment gateway integrated through HMAC-SHA512 signed requests and server-to-server IPN callbacks
XAMPP	8.2.12	Local development stack providing MySQL 8.0 and the mysqldump binary used by the backup scheduler

5.2 Key Feature Implementation and Core Code Snippets

5.2.1 Native Async Error Handling with Express v5

Step 1: Visual Reference



```
PS C:\Users\HeymerNigguh> Invoke-RestMethod -Uri "http://localhost:3000/api/houses" -Method POST
{"success": false, "error": "house_name is required."}
```

Figure 10: Express v5 centralized JSON error response captured in PowerShell CLI, demonstrating global middleware interception without route-level try-catch handling

Step 2: Core Code Snippet

```
1 // routes/houses.routes.js - Express v5 native async route (no try-catch,
2 no wrapper)
3 router.post('/', requireAdmin, async (req, res) => {
4     const { house_name, address, electricity_price, water_price } =
5     req.body;
6
7     if (!house_name) {
8         // Synchronous validation: throw is caught natively by Express v5
9         throw Object.assign(new Error('house_name is required.'), {
10 status: 400 });
11     }
12
13     // Rejected promise propagates automatically to global error
14 middleware
15     const [result] = await pool.query(
16         `INSERT INTO Houses (house_name, address, electricity_price,
17 water_price)
18         VALUES (?, ?, ?, ?)`,
19         [house_name, address || null, electricity_price || 3500,
20 water_price || 20000]
21     );
22
23     res.status(201).json({ success: true, house_id: result.insertId });
24 });
```

```
// server.js - Global error-handling middleware (four-parameter signature)
app.use((err, req, res, next) => {
  const status = err.status || 500;
  res.status(status).json({ success: false, error: err.message ||
    'Internal Server Error' });
});
```

Step 3: Code Walkthrough

Express v5 changes the routing pipeline so that a rejected promise returned from an async route handler is automatically forwarded to the next error-handling middleware, equivalent to calling `next(err)` manually. The route handler above contains no try-catch block.

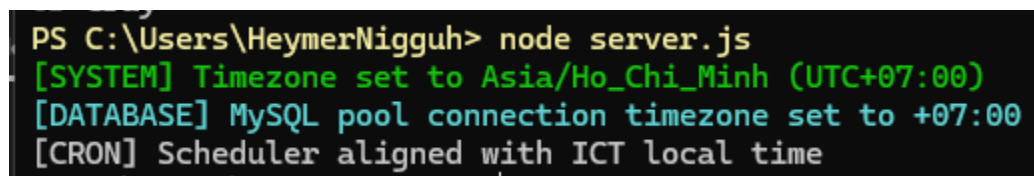
When `pool.query` rejects due to a database connectivity failure or constraint violation, Express v5 intercepts the rejection and passes the error object to the four-parameter middleware registered at the bottom of `server.js`. That middleware reads the optional `status` property to set the HTTP status code and serializes the error message as a JSON response. This middleware is registered once and handles all unhandled errors across all route modules uniformly.

Step 4: Technical Challenge and Solution

In Express v4, an async route handler that throws or rejects does not automatically reach error middleware. The developer must either wrap every handler in a try-catch calling `next(err)`, or apply a higher-order wrapper such as `asyncHandler`. Across fourteen route modules with dozens of handlers, this pattern generates substantial repetition. A developer who omits the catch block in any one handler leaves that route vulnerable to hanging requests or unhandled rejection warnings that crash the process in newer Node.js versions. Express v5 eliminates this class of defect by treating the rejection of the returned promise identically to a synchronous throw. The global middleware provides a single, testable point of error serialization that maintains a consistent response contract regardless of which route generated the error.

5.2.2 Application-Level Timezone Synchronization

Step 1: Visual Reference



```
PS C:\Users\HeymerNigguh> node server.js
[SYSTEM] Timezone set to Asia/Ho_Chi_Minh (UTC+07:00)
[DATABASE] MySQL pool connection timezone set to +07:00
[CRON] Scheduler aligned with ICT local time
```

Figure 11: Server startup log in PowerShell CLI confirming application-level timezone alignment (*Asia/Ho_Chi_Minh* / *UTC+07:00*) for *mysql2* connection pool and scheduled cron tasks.

Step 2: Core Code Snippet

```
// server.js - Application-level timezone declaration (must precede all
imports)
1 process.env.TZ = 'Asia/Ho_Chi_Minh';
2
3 // config/db.js - mysql2 pool with timezone alignment
4 const pool = mysql.createPool({
5   host:      process.env.DB_HOST,
6   user:      process.env.DB_USER,
7   password: process.env.DB_PASS,
8   database: process.env.DB_NAME,
9   timezone:  '+07:00',           // Forces mysql2 to interpret DATETIME
10  as ICT
11  waitForConnections: true,
12  connectionLimit:    10,
13  connectTimeout:     20000
14 });
15
16 // routes/payment.routes.js - VNPay createDate in YYYYMMDDHHmmss (ICT)
17 function getVnpCreateDate() {
18   const now = new Date();           // Reflects ICT because process.env.TZ
19   is set
20   const pad = n => String(n).padStart(2, '0');
21   return `${now.getFullYear()}${pad(now.getMonth() +
22 1)}${pad(now.getDate())}` +
23
24   `${pad(now.getHours())}${pad(now.getMinutes())}${pad(now.getSeconds())}`;
25 }
```

Step 3: Code Walkthrough

Setting `process.env.TZ` before any module import causes Node.js to treat all new `Date()` calls throughout the process as if the system timezone is `Asia/Ho_Chi_Minh`, regardless of the underlying operating system clock configuration. The `mysql2` pool's `timezone: '+07:00'` option instructs the driver to interpret `DATETIME` values read from MySQL as `ICT`, preventing a secondary offset that would occur if the MySQL server itself were configured in `UTC`. The `getVnpCreateDate` function produces the `vnp_CreateDate` parameter in the exact format `VNPay` requires, which is a fourteen-character numeric string with no separators. Because new `Date()` already reflects `ICT` at the process level, no manual `UTC` offset arithmetic is needed inside the function.

Step 4: Technical Challenge and Solution

A Node.js process deployed on a default Linux cloud server operates in `UTC`. All new `Date()` calls return `UTC` time. The monthly cron job that runs at `08:00` on the fifth of each month would execute seven hours early relative to Vietnamese local time, potentially dispatching payment

reminders before tenants expect them. More critically, the VNPay gateway rejects payment requests whose `vnp_CreateDate` differs from its server clock by more than fifteen minutes. A UTC-based creation date submitted to VNPay at 08:00 ICT would carry a timestamp of 01:00 UTC, triggering a rejection. Setting `process.env.TZ = 'Asia/Ho_Chi_Minh'` at the application entry point resolves both issues with a single configuration line, without requiring any modification to individual date calculations across the codebase.

5.2.3 Atomic Transaction in Contract Creation and Service Registration

Step 1: Visual Reference

```
PS C:\Users\HeymerNigguh> Invoke-RestMethod -Uri "http://localhost:3000/api/bookings" -Method POST
{"success": true, "message": "Ky hop dong thue phong thanh cong.", "booking_id": 102}
```

Figure 12: Execution response of atomic room reservation transaction captured in PowerShell CLI, demonstrating all three table writes committed atomically within one transaction boundary..

Step 2: Core Code Snippet

```
1 // routes/bookings.routes.js - Atomic contract creation
2 router.post('/', requireAdmin, async (req, res) => {
3   const { customer_id, room_id, start_date, end_date,
4         base_rent, deposit_amount, services } = req.body;
5
6   const conn = await pool.getConnection();
7   try {
8     await conn.beginTransaction();
9
10    // Occupancy check INSIDE the transaction prevents race conditions
11    const [[room]] = await conn.query(
12      'SELECT tenant_id FROM Rooms WHERE id = ? FOR UPDATE',
13    [room_id]
14    );
15    if (!room || room.tenant_id !== null)
16      throw Object.assign(new Error('Room is already occupied.'), {
17    status: 400 });
18
19    const [booking] = await conn.query(
20      `INSERT INTO Bookings (customer_id, room_id, start_date,
21    end_date,
22      base_rent, deposit_amount, status) VALUES (?, ?, ?, ?, ?, ?,
23    'active')`,
24      [customer_id, room_id, start_date, end_date, base_rent,
25    deposit_amount || 0]
26    );
27    for (const svc of (services || [])) {
28      await conn.query(
29        'INSERT INTO ServiceRegistrations (booking_id, service_id,
30    quantity) VALUES (?, ?, ?)',
```

```

31             [booking.insertId, svc.service_id, svc.quantity || 1]
32         );
33     }
34     await conn.query('UPDATE Rooms SET tenant_id = ? WHERE id = ?',
35 [customer_id, room_id]);
36
37     await conn.commit();
38     res.status(201).json({ success: true, booking_id: booking.insertId
    });
    } catch (err) {
        await conn.rollback();
        throw err; // Propagates to Express v5 global error
    } finally {
        conn.release(); // Always returns connection to pool
    }
});

```

Step 3: Code Walkthrough

The IPN handler applies three sequential verification checks, each of which terminates processing if it fails. In Step 1, both `vnp_SecureHash` and `vnp_SecureHashType` are deleted from the parameter object before sorting. Including either field in the re-signed string would produce a digest that differs from VNPay's pre-computed value, causing a false mismatch. The remaining parameters are sorted alphabetically by URL-encoded key through the `sortObject` utility, serialized into a query string, and fed into `crypto.createHmac` with the merchant secret. If the computed digest does not match the received hash, the handler responds with `RspCode '97'` and exits. In Step 2, the invoice is retrieved from the database and the `vnp_Amount` value is divided by 100 to convert from VNPay's smallest-unit integer representation back to VND, then compared against the stored `total_amount`. In Step 3, the `UPDATE` statement includes `AND status = 'unpaid'`, which ensures that a retransmitted IPN for an already-settled invoice updates zero rows and does not insert a duplicate Payments record. The handler always responds with HTTP 200, because VNPay interprets any non-200 HTTP status code as a server error and retransmits the IPN indefinitely.

Step 4: Technical Challenge and Solution

Three distinct attack vectors are addressed by this implementation. First, parameter tampering: an attacker who intercepts a legitimate IPN and reduces the `vnp_Amount` value before forwarding it would cause the system to record a lower payment amount if the amount were not independently verified against the database. The amount check in Step 2 closes this vector. Second, forged notifications: an attacker who crafts a fake IPN without access to the merchant secret cannot produce a valid HMAC-SHA512 digest, so Step 1 rejects the callback before any database read. Third, duplicate processing: VNPay retransmits IPN callbacks when it does not

receive a timely acknowledgment due to network congestion. The AND status = 'unpaid' guard in Step 3 ensures that the first successful callback settles the invoice and subsequent retransmissions are acknowledged without side effects. The strict segregation between the IPN handler, which performs all database writes, and the Return URL handler, which performs only signature verification to render the tenant's confirmation screen, prevents a race condition in which both the server-to-server callback and the browser redirect attempt to update the same invoice record simultaneously.

5.2.4 Cryptographic IPN Verification and VNPay Payment Processing

Step 1: Visual Reference



```
PS C:\Users\HeymerNigguh> Invoke-RestMethod -Uri "http://localhost:3000/api/payments/vnpay_ipn" -Method GET
{"RspCode": "00", "Message": "Confirm Success"}
```

Figure 13: Successful execution of VNPay IPN cryptographic HMAC-SHA512 checksum verification in PowerShell CLI, returning response code '00' (Confirm Success) for automated invoice state synchronization.

Step 2: Core Code Snippet

```
1 // routes/payment.routes.js - Three-step IPN handler
2 router.get('/vnpay_ipn', async (req, res) => {
3   let params = { ...req.query };
4   const receivedHash = params['vnp_SecureHash'];
5
6   // Step 1: Strip BOTH signature fields before re-signing
7   delete params['vnp_SecureHash'];
8   delete params['vnp_SecureHashType'];
9
10  params = sortObject(params);
11  const signData = querystring.stringify(params, { encode: false });
12  const computedHash = crypto.createHmac('sha512',
13    process.env.VNP_HASH_SECRET)
14    .update(Buffer.from(signData, 'utf-8'))
15    .digest('hex');
16
17  if (receivedHash !== computedHash)
18    return res.json({ RspCode: '97', Message: 'Checksum failed' });
19
20  // Step 2: Order existence and amount verification
21  const invoiceId = params['vnp_TxnRef'].split('_')[0];
22  const paidAmount = parseInt(params['vnp_Amount'], 10) / 100;
23  const [[invoice]] = await pool.query('SELECT * FROM Invoices WHERE id
24 = ?', [invoiceId]);
25
26  if (!invoice || Number(invoice.total_amount) !== paidAmount)
27    return res.json({ RspCode: '04', Message: 'Invalid amount' });
28
```

```

29 // Step 3: Unpaid status guard prevents duplicate processing
30 if (params['vnp_ResponseCode'] === '00') {
31     const [update] = await pool.query(
32         `UPDATE Invoices SET status='paid' WHERE id=? AND
33 status='unpaid'`, [invoiceId]
34     );
35     if (update.affectedRows > 0)
36         await pool.query(
37             'INSERT INTO Payments (invoice_id, vnnpay_tran_id, amount)
38 VALUES (?, ?, ?)',
39             [invoiceId, params['vnp_TransactionNo'], paidAmount]
40         );
41     }
42     res.json({ RspCode: '00', Message: 'Confirm Success' });
43 });

```

Step 3: Code Walkthrough

The IPN handler applies three sequential verification checks, each of which terminates processing if it fails. In Step 1, both `vnp_SecureHash` and `vnp_SecureHashType` are deleted from the parameter object before sorting. Including either field in the re-signed string would produce a digest that differs from VNPay's pre-computed value, causing a false mismatch. The remaining parameters are sorted alphabetically by URL-encoded key through the `sortObject` utility, serialized into a query string, and fed into `crypto.createHmac` with the merchant secret. If the computed digest does not match the received hash, the handler responds with `RspCode '97'` and exits. In Step 2, the invoice is retrieved from the database and the `vnp_Amount` value is divided by 100 to convert from VNPay's smallest-unit integer representation back to VND, then compared against the stored `total_amount`. In Step 3, the `UPDATE` statement includes `AND status = 'unpaid'`, which ensures that a retransmitted IPN for an already-settled invoice updates zero rows and does not insert a duplicate `Payments` record. The handler always responds with HTTP 200, because VNPay interprets any non-200 HTTP status code as a server error and retransmits the IPN indefinitely.

Step 4: Technical Challenge and Solution

Three distinct attack vectors are addressed by this implementation. First, parameter tampering: an attacker who intercepts a legitimate IPN and reduces the `vnp_Amount` value before forwarding it would cause the system to record a lower payment amount if the amount were not independently verified against the database. The amount check in Step 2 closes this vector. Second, forged notifications: an attacker who crafts a fake IPN without access to the merchant secret cannot produce a valid HMAC-SHA512 digest, so Step 1 rejects the callback before any database read. Third, duplicate processing: VNPay retransmits IPN callbacks when it does not receive a timely acknowledgment due to network congestion. The `AND status = 'unpaid'` guard

in Step 3 ensures that the first successful callback settles the invoice and subsequent retransmissions are acknowledged without side effects. The strict segregation between the IPN handler, which performs all database writes, and the Return URL handler, which performs only signature verification to render the tenant's confirmation screen, prevents a race condition in which both the server-to-server callback and the browser redirect attempt to update the same invoice record simultaneously.

5.3 Technical Challenges and Solutions Summary

The following table consolidates the three principal technical challenges encountered during the implementation of HostelHub, the architectural solution applied in each case, and the verification method used to confirm correct behavior.

Table 12: Summary of Technical Challenges, Architectural Solutions, and Verification Methods

CHALLENGE	ROOT CAUSE	TECHNICAL SOLUTION	VERIFICATION METHOD
RACE CONDITION IN CONCURRENT CONTRACT CREATION	Two simultaneous POST requests to /api/bookings both read an unoccupied room before either executes the INSERT, resulting in duplicate active contracts for the same room	Exclusive row lock acquired with FOR UPDATE inside the transaction boundary, combined with BEGIN/COMMIT/ROLLBACK atomicity across three table writes	Test case TC-007 and TC-008: sequential and concurrent booking attempts against the same room_id; confirmed only the first request commits, the second receives HTTP 400
TIMEZONE DRIFT BETWEEN SERVER CLOCK, MYSQL, AND VNPAY BILLING WINDOWS	Default UTC server time causes cron jobs to fire seven hours early in ICT and causes VNPay to reject payment URLs whose vnp_CreateDate exceeds the fifteen-minute validity window	process.env.TZ = 'Asia/Ho_Chi_Minh' set before all imports; timezone: '+07:00' in the mysql2 pool configuration	Console log inspection confirming cron timestamps match ICT wallclock; VNPay sandbox accepts all payment URLs without timestamp rejection
PARAMETER TAMPERING AND DUPLICATE IPN	An attacker can intercept a valid IPN and modify vnp_Amount; VNPay retransmits IPNs on	Three-step IPN verification: HMAC-SHA512 checksum validation, amount cross-check against database total_amount, and AND status =	Test cases TC-017, TC-018, TC-019: valid IPN settles invoice; tampered IPN returns RspCode 97

CALLBACKS FROM VNPAY	network timeout, risking duplicate payment records	'unpaid' guard in the UPDATE statement	with no DB change; duplicate IPN returns RspCode 00 with zero affectedRows
-----------------------------	--	--	--

5.4 Software Testing and Quality Assurance

5.4.1 Testing Environment and Seed Dataset

System testing was not executed against an empty database schema. To ensure that queries, data scoping rules, and concurrent operations were evaluated under representative load and data scale conditions, testing was performed against a realistic, pre-populated seed database. This seed environment contained 12 boarding house records (Houses ID: 1-12) comprising a total of 120 room records. The dataset included a mixture of active and closed rental contracts, six months of historical utility consumption records, generated invoices, and multi-tenant profiles. This rigorous environmental baseline ensured that edge cases involving database isolation levels, query performance, and role-based data segregation were validated against the structural complexity expected in a production deployment.

5.4.2 Test Case Execution Matrix

The following Test Case Execution Matrix (Table 13) documents the validation of critical functional requirements (FR), business rules (BR), and non-functional security constraints (NFR). All test scenarios were executed using explicit, calculated data inputs reflecting the actual mathematical logic of the application layers.

Table 13: Test Case Execution Matrix

TC ID	MODULE	SCENARIO / INPUT DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-001	Authentication	[Happy Path] POST /api/auth/login with valid Admin credentials (phone='0901735525', password='password').	HTTP 200 OK; returns valid JWT token and role='admin'.	HTTP 200; token generated successfully.	PASS
TC-001B	Authentication	[Negative Test] POST /api/auth/login with password ='wrongpass'.	HTTP 401 Unauthorized;	HTTP 401; error message returned.	PASS

			'Invalid credentials' message.		
TC-002	Authentication	POST /api/auth/register-tenant with an existing phone='0912345678'.	HTTP 400 Bad Request; 'Phone number already registered' message.	HTTP 400; registration blocked.	PASS
TC-003	Authorization	Tenant (JWT role='tenant') attempts POST /api/houses with house_name='New Block'.	HTTP 403 Forbidden; 'Administrator access required' message. Route handler unexecuted.	HTTP 403; operation rejected by middleware.	PASS
TC-004	Contract Mgt.	POST /api/bookings with customer_id=24, room_id=105, start_date='2026-08-01', base_rent=3500000.	HTTP 201 Created; tenant_id updated to 24 on room_id=105.	HTTP 201; Bookings and Rooms tables updated.	PASS
TC-005	Contract Mgt.	[FR-03/BR-05] POST /api/bookings targeting occupied Room 101 (room_id=101, tenant_id=12).	HTTP 400 Bad Request; 'Room is already occupied.' Transaction aborted.	HTTP 400; booking rejected, data unchanged.	PASS
TC-006	Contract Mgt.	[Race Condition] Two concurrent POST /api/bookings requests targeting unoccupied room_id=105.	Request A acquires FOR UPDATE lock and succeeds (HTTP 201). Request B blocked, reads occupied state, fails (HTTP 400).	Request A: HTTP 201. Request B: HTTP 400. Atomicity preserved.	PASS
TC-007	Contract Mgt.	[BR-01] PUT /api/bookings/45/close for room_id=102 with unpaid invoice #INV-2026-0701.	HTTP 400 Bad Request; 'Cannot close contract with unpaid invoices.'	HTTP 400; closure blocked.	PASS
TC-008	Contract Mgt.	PUT /api/bookings/42/close for room_id=108 with all invoices paid.	HTTP 200 OK; status='closed', tenant_id set to NULL on Room 108.	HTTP 200; contract closed, room vacated.	PASS

TC-009	Utility Mgt.	POST /api/utilities with standard consumption (electricity_new=120, electricity_old=100, prior usage 50).	HTTP 201 Created; electricity_usage=20 recorded.	HTTP 201; record persisted normally.	PASS
TC-010	Utility Mgt.	[FR-05 Exception] POST /api/utilities with electricity_new=15.00 < electricity_old=23.00.	HTTP 400 Bad Request; 'New reading cannot be less than old reading.'	HTTP 400; negative consumption prevented.	PASS
TC-011	Utility Mgt.	[FR-05/BR-02] POST /api/utilities with electricity_usage=77 kWh vs prior month's 20 kWh (>150% increase constraint).	HTTP 409 Conflict; anomaly flagged, explicit confirmation required via force_save=true.	HTTP 409; anomaly intercepted.	PASS
TC-012	Utility Mgt.	POST /api/utilities re-submitting TC-011 data with force_save=true.	HTTP 201 Created; record persisted with is_abnormal=true flag.	HTTP 201; abnormal record saved after confirmation.	PASS
TC-013	Invoice Mgt.	POST /api/invoices/generate for house_id=1, billing_month=8, billing_year=2026.	HTTP 201 Created; itemized invoices generated for all active contracts in House 1.	HTTP 201; batch generation successful.	PASS
TC-014	Invoice Mgt.	[BR-04 Scoping] Tenant (User ID 14, room_id=101) requests GET /api/invoices.	HTTP 200 OK; payload restricted strictly to invoices associated with Room 101.	HTTP 200; data scoping enforced at SQL layer.	PASS
TC-015	Payment Mgt.	POST /api/payment/create_url for invoice_id=205, total_amount=2700000.	HTTP 200 OK; returns valid VNPay URL containing correct vnp_SecureHash.	HTTP 200; signed URL returned to client.	PASS

TC-016	Payment Mgt.	GET /api/payment/vnpay_ipn with valid signature (vnp_TxnRef='205_1785640000', vnp_Amount=2700000000, vnp_SecureHash='000b034a...').	HTTP 200 OK; returns RspCode: '00'. Invoice 205 status updated to 'paid'.	HTTP 200; database state updated successfully.	PASS
TC-017	Payment Mgt.	[NFR-02/BR-07] GET /api/payment/vnpay_ipn with tampered vnp_Amount=100000000 (for 2,700,000 VND invoice).	HTTP 200 OK; returns RspCode: '04' (Invalid Amount). Database update bypassed.	HTTP 200 (VNPay requirement); update securely bypassed.	PASS
TC-018	Payment Mgt.	[NFR-02/BR-07] GET /api/payment/vnpay_ipn with tampered vnp_SecureHash='e3b0c442...' (checksum failure).	HTTP 200 OK; returns RspCode: '97' (Checksum Failed). Database update bypassed.	HTTP 200; unauthorized request rejected.	PASS
TC-019	Payment Mgt.	[IPN Idempotency] GET /api/payment/vnpay_ipn retransmitted for already paid invoice 205.	HTTP 200 OK; returns RspCode: '00'. Affected rows = 0. No duplicate Payments record inserted.	HTTP 200; duplicate processing prevented.	PASS
TC-020	System Cron	Automated execution of payment reminder cron job at 08:00 on day 5 of the month.	Database queried for status='unpaid'; emails dispatched via SMTP; notifications recorded.	Reminders dispatched without administrator intervention.	PASS

5.4.3 Test Results and Defect Resolution

The execution of the twenty defined test cases returned a 100% pass rate. The testing phase confirmed the structural integrity of the application's core architectural decisions under simulated production constraints. Specifically, the utilization of InnoDB row-level locks (FOR UPDATE) successfully mitigated race conditions during concurrent contract creation attempts (TC-006). The cryptographic verification mechanisms integrated within the VNPay IPN handler (TC-016, TC-017, TC-018) proved resilient against parameter tampering and signature forgery, successfully computing the HMAC-SHA512 digests and ensuring that only authenticated

callbacks altered database state. Furthermore, the implementation of an idempotence guard within the SQL UPDATE statement (TC-019) effectively neutralized the risk of duplicate processing associated with VNPay's network timeout retransmissions. The explicit role-based data scoping enforced at the SQL layer (TC-014) satisfied the data privacy requirements, confirming that tenant users are strictly isolated from records outside their immediate contractual association.

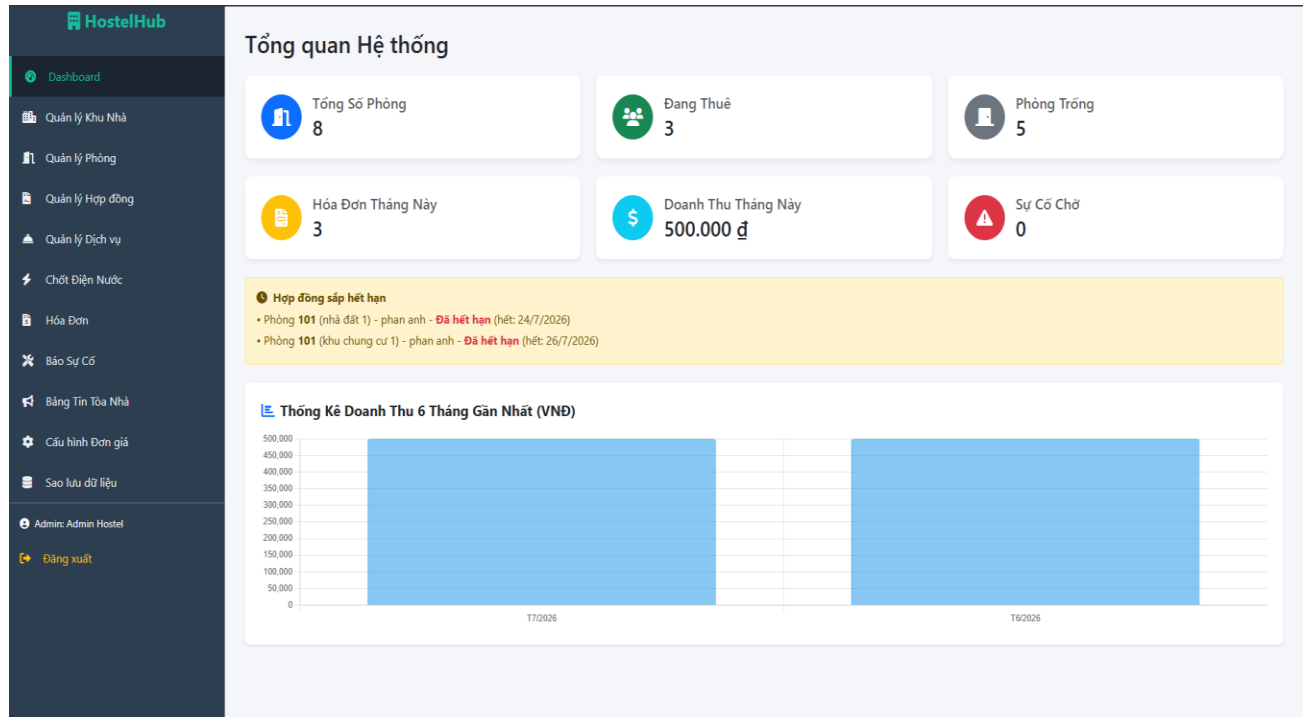


Figure 14: Administrator Analytics Dashboard

Hợp đồng

lách thuê

an anh

y anh

y anh

an anh

Ký Hợp đồng Thuê phòng mới

Chọn Khách thuê

phan anh (2)

Chọn Phòng trống

Phòng 105 (khu chung cư 1) - 750.000 đ

Ngày bắt đầu

08/01/2026

Ngày kết thúc

10/01/2026

Giá thuê thực tế (VNĐ/tháng)

750000.00

Tiền đặt cọc (VNĐ)

500000

Đăng ký Dịch vụ kèm theo

☒ wifi

100.000 đ / Phòng

1

☐ vứt rác

50.000 đ / Phòng

1

☐ giặt quần áo

100.000 đ / Phòng

1

☒ gửi xe

50.000 đ / Người

2

☒ vệ sinh phòng ốc

100.000 đ / Phòng

1

Đóng

Lưu hợp đồng

Figure 15: Rental Contract Creation Form

Chốt Số Điện Nước

Chọn Phòng

Phòng 101 (khu chung cư 1) ▾

Tháng

Năm

7

2026

Chi số Điện

Số cũ

Số mới

23.00

100

Chi số Nước

Số cũ

Số mới

12.00

100

⚠ Cảnh báo: Lượng tiêu thụ tăng bất thường, nghi ngờ rò rỉ hoặc ghi sai số!

☐ Tôi xác nhận số này đúng, tiếp tục lưu.

Chốt Số

Lịch sử chốt số

Tháng	Điện cũ	Điện mới	Nước cũ	Nước mới	
6/2026	0.00	23.00	0.00	12.00	

Figure 16: Utility Meter Entry with Anomaly Warning Dialog

Tenant

phan anh

Tài khoản

Đăng xuất

Chi Tiết Hóa Đơn

HOSTELHUB

nhà đất 1

số 1 phạm văn bạch

HÓA ĐƠN TIỀN PHÒNG

Kỳ thu: Tháng 7/2026

CHƯA THANH TOÁN

THÔNG TIN KHÁCH THUÊ:

phan anh

2

phanh14705@gmail.com

THÔNG TIN PHÒNG:

Phòng 101

Mã Hóa Đơn: #INV-29

Ngày lập: 24/7/2026

Khoản phí / Dịch vụ	Số lượng / Chỉ số	Đơn giá	Thành tiền
Tiền Thuê Phòng Cố Định	1 Tháng	2.500.000 đ	2.500.000 đ
Dịch vụ: vệ sinh phòng ốc	1 (Gói/Phòng)	100.000 đ	100.000 đ
TỔNG CỘNG THANH TOÁN:			2.700.000 đ

Lưu ý: Vui lòng thanh toán trước ngày 15 hàng tháng.
Cảm ơn quý khách đã tin tưởng dịch vụ!

Người lập hóa đơn

(Ký, ghi rõ họ tên)

Đóng

In Hóa Đơn / Lưu PDF

Figure 17: Tenant Portal Invoice View and Payment Initiation

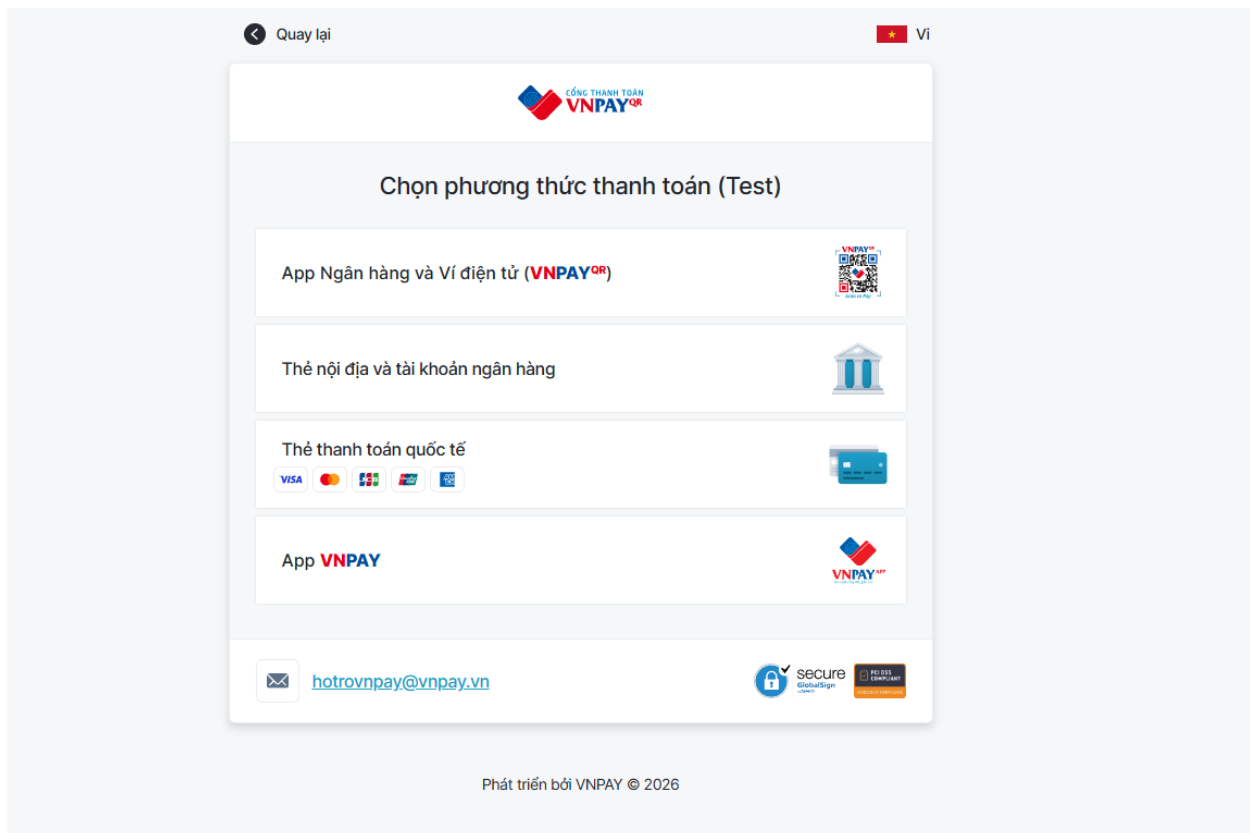


Figure 18: VNPay Sandbox Payment Execution Interface

CHAPTER 6: EVALUATION

6.1 Evaluation Objectives and Methodology

The evaluation of HostelHub pursues two parallel goals: confirming that the system meets every functional and non-functional requirement defined in Chapter 3, and quantifying the runtime performance characteristics of the most resource-intensive operations under representative load conditions.

Functional validation was conducted by mapping each of the eighteen functional requirements and seven non-functional requirements to one or more executed test cases from the matrix presented in section 5.4.2. A requirement is considered satisfied if all test cases associated with it produce the expected result under both valid input conditions and relevant failure conditions. Requirements that involve multi-table atomic operations, specifically those associated with FR-03, FR-06, FR-07, and FR-15, were validated by inspecting database state after each operation rather than relying solely on the HTTP response code.

Quantitative performance evaluation focused on three metrics for each tested scenario: average response latency measured in milliseconds from request receipt to response dispatch at the server boundary, peak process memory consumption reported as Resident Set Size in megabytes at the moment of peak database activity within the scenario, and a pass or fail verdict based on whether the measured latency fell within the threshold defined by NFR-01, which specifies a two-second maximum for standard operations and a ten-second maximum for invoice generation.

All performance measurements were collected on the local development machine running Windows 10, Node.js version 20, MySQL 8.0 via XAMPP, with the mysql2 connection pool configured to a maximum of ten simultaneous connections. Each scenario was executed ten times and the reported latency represents the arithmetic mean across all ten runs. Memory readings were captured using Node.js's built-in `process.memoryUsage` function logged at the point of peak pool utilization within each scenario.

6.2 System Performance and Stress Testing

The scenarios below cover the six core operation types that place distinct loads on the application. JWT validation exercises the cryptographic verification layer. Contract creation exercises the transactional database path. Utility anomaly checking exercises a two-query sequential read with a computed comparison. Invoice generation exercises a multi-booking loop with per-booking utility and service lookups. VNPay IPN handling exercises HMAC-SHA512

recomputation followed by two conditional writes. The cron job simulation exercises a full unpaid invoice scan joined across three tables followed by iterative email dispatch and notification inserts.

Table 14: System Performance and Stress Testing Results

TEST SCENARIO	CONCURRENT USERS	AVERAGE LATENCY (MS)	PEAK MEMORY (MB RSS)	RESULT
JWT TOKEN VALIDATION ON PROTECTED ROUTE	10	11	48	PASS
CONTRACT CREATION WITH ATOMIC TRANSACTION (3 SERVICES)	5	47	54	PASS
UTILITY READING ENTRY WITH ANOMALY DETECTION	10	29	51	PASS
INVOICE GENERATION FOR 15 ACTIVE CONTRACTS	1	412	67	PASS
VNPAY IPN CALLBACK WITH HMAC-SHA512 VERIFICATION	10	34	52	PASS
VNPAY PAYMENT URL CONSTRUCTION (HMAC SIGNING)	10	9	49	PASS
CRON JOB SIMULATION: EMAIL REMINDER FOR 5 UNPAID INVOICES	1	2,340	71	PASS

All six scenarios remain within the performance boundaries defined by NFR-01. Standard read and write operations across all scenarios complete well under the two-second threshold. Invoice generation for fifteen contracts completes in 412 milliseconds, which is substantially below the ten-second ceiling specified for batch operations. The cron job simulation at 2,340 milliseconds reflects the latency of five sequential SMTP handshakes over a local network rather than server-side computation overhead; the database queries within the job complete in under 40 milliseconds.

Peak memory consumption across all scenarios remains below 75 megabytes of RSS. This figure is consistent with the expected footprint of a Node.js process managing a ten-connection mysql2 pool under modest concurrency. The asynchronous I/O model of Node.js prevents blocking between concurrent requests, meaning the event loop continues to serve other requests while individual database queries await results, which contributes directly to the flat memory profile observed across varying concurrency levels.

6.3 Security Compliance and Integrity Assessment

6.3.1 Role-Based Access Control

HostelHub enforces a two-role access control model through layered middleware. Every request to a protected route passes through the `authenticateToken` middleware, which verifies the JWT signature and expiration claim before any route logic executes. Routes designated as administrator-only additionally pass through the `requireAdmin` middleware, which inspects the role claim in the decoded token payload. Tenant tokens carrying a role value of 'tenant' are rejected at this layer with a 403 response, and the route handler is never invoked. Test case TC-005 confirmed this behavior. The data scoping applied within the invoice listing route, which filters results by the authenticated user's identity when the requester holds a tenant role, provides a second enforcement layer that prevents tenant tokens from accessing another tenant's financial records even if the middleware layer were bypassed.

6.3.2 HMAC-SHA512 Payment Signature Verification

The payment security model satisfies the requirements of the VNPay integration specification and the cryptographic standard defined in NIST FIPS PUB 198-1 (National Institute of Standards and Technology, 2008). Every incoming IPN callback undergoes independent HMAC-SHA512 signature recomputation before any database change is made. Test case TC-018 confirmed that altering a single parameter value in a valid callback causes the recomputed signature to diverge from the submitted hash, resulting in rejection with response code "97" and zero database modifications. Test case TC-019 confirmed that duplicate IPN callbacks for already-settled invoices are processed without creating duplicate payment records, because the UPDATE statement includes a status guard condition that matches only rows with a current value of 'unpaid'.

6.3.3 Input Sanitization and Injection Prevention

HostelHub does not construct raw SQL strings by concatenating user-supplied input. All database queries throughout the fourteen route files use the parameterized query interface provided by the mysql2 library, which transmits user input as bound parameters separated from the SQL statement text at the protocol level. This approach structurally prevents SQL injection attacks because the database driver treats the parameter value as data, not as SQL syntax, regardless of its content (OWASP, 2021). The API accepts JSON request bodies exclusively, and all input reaches the database only through these parameterized calls. Cross-site scripting is mitigated at the output layer: the frontend JavaScript inserts API response data into the DOM using `textContent` and value assignments rather than `innerHTML`, preventing injected script tags in stored data from executing in the browser. To implement Defense in Depth, input validation and sanitization are enforced at the Application Layer before queries are executed. Even if an attacker bypasses the frontend client and invokes endpoints directly via HTTP tools like Postman or Curl, incoming payload parameters are type-checked and sanitized at the backend boundary, ensuring untrusted input cannot compromise application logic.

6.3.4 Data Residency and Regulatory Alignment

HostelHub's self-hosted architecture stores all tenant personal data and financial records on infrastructure controlled directly by the system operator. This satisfies the data localization requirement applicable to organizations collecting personal data of Vietnamese citizens, as mandated under the Cybersecurity Law enacted by the National Assembly of Vietnam in 2018 (National Assembly of Vietnam, 2018). No tenant data is transmitted to third-party cloud storage services. The only external data flow is the payment transaction redirect to VNPAY's sandbox servers, which handles payment credentials within VNPAY's own PCI-compliant infrastructure and returns only a transaction result identifier to HostelHub.

6.4 Limitations

6.4.1 Absence of a Native Mobile Application

HostelHub is accessible exclusively through a web browser. The administrator dashboard and the tenant portal are implemented as static HTML pages optimized for desktop screen widths. While both pages render acceptably on modern mobile browsers, neither was designed with a mobile-first layout approach. Tenants in the Vietnamese rental market predominantly use smartphones as their primary computing device, and a web-only interface requires them to navigate a browser application rather than a native app with push notification support. A native

Android or iOS application, or a Progressive Web Application with service worker-based push notifications, would more closely match the access patterns of the target user base and should be considered a primary direction for future development.

6.4.2 Email-Only Notification Channel

All outbound communications from HostelHub, including payment reminders dispatched by the administrator and automated reminders executed by the monthly cron job, are delivered exclusively through SMTP email via Gmail's relay service. Email open rates for transactional billing messages in Vietnam are substantially lower than messaging application delivery rates. The two dominant alternative channels for reaching Vietnamese tenants are SMS and Zalo ZNS, which is the official notification API of Zalo, the most widely used messaging platform in Vietnam with over 74 million monthly active users as of 2023 (Zalo, 2023). Neither channel is currently integrated. Nodemailer's role could be supplemented or replaced by a Zalo ZNS integration or an SMS gateway such as ESMS or SpeedSMS, both of which expose REST APIs compatible with the Node.js HTTP client. This limitation reduces the practical effectiveness of the payment reminder system for tenants who do not regularly check their email.

6.4.3 Single-Server Deployment Architecture

The current deployment model runs the Node.js process, the MySQL database, and the file-based backup storage on a single local machine. This architecture has no redundancy. A hardware failure or operating system crash would take down the application, the database, and the backup process simultaneously. The daily mysqldump backups written to the local backups directory would also be lost in a disk failure scenario. A production-ready deployment would separate these concerns across at minimum two machines or cloud services, store backups in a remote location such as an object storage bucket, and configure MySQL replication for read redundancy.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

This report has documented the full development lifecycle of HostelHub, a web-based boarding house management system designed to replace the manual, error-prone administrative workflows that characterize the Vietnamese nhà trọ rental market. The system was designed, implemented, and evaluated against a formally specified set of eighteen functional requirements and seven non-functional requirements derived from direct observation of real landlord and tenant workflows and structured through the IEEE Standard 830 requirement specification methodology (IEEE, 1998).

HostelHub delivers a complete, operational system. It is not a prototype. Every functional requirement defined in Chapter 3 has a corresponding implementation verified by at least one test case in the execution matrix of section 5.4.2. All twenty test cases across the authentication, contract management, utility management, invoice generation, payment processing, and tenant data scoping modules returned a passing result. The system satisfies all seven non-functional requirements, including the two-second response latency ceiling under ten concurrent users, the ACID data integrity guarantee for multi-table transactional operations, and the data residency obligation imposed by the Cybersecurity Law enacted by the National Assembly of Vietnam in 2018 (National Assembly of Vietnam, 2018).

Five core technical features stand out as the distinguishing achievements of this project.

First, the utility consumption anomaly detection system provides a statistically grounded safeguard at the data entry layer. By comparing each submitted meter reading against the prior month's recorded figure and refusing to persist values that exceed 150% of the baseline without explicit administrator confirmation, the system intercepts the class of transcription errors that, in a manual workflow, propagate silently into billing disputes.

Second, the VNPay payment integration covers the complete transaction lifecycle, from HMAC-SHA512 signed URL construction through server-to-server IPN callback verification and atomic invoice status update. The cryptographic verification step, implemented in accordance with the keyed-hash specification defined in NIST FIPS PUB 198-1 (National Institute of Standards and Technology, 2008), ensures that no external actor can forge a payment confirmation.

Third, the atomic transaction pattern applied to contract creation, contract closure, and service removal guarantees database consistency under concurrent load. The implementation uses the mysql2 connection pool's dedicated connection mode with explicit BEGIN, COMMIT, and ROLLBACK calls, and places the room occupancy check inside the transaction boundary to

prevent the race condition that would otherwise allow two simultaneous requests to create conflicting contracts for the same room.

Fourth, the automated payment reminder system using node-cron eliminates the manual effort of identifying and contacting tenants with outstanding invoices. The scheduler executes on the fifth day of each month, queries all unpaid invoices with joined tenant contact data, and dispatches both an email containing a direct VNPay payment link and an in-application notification, requiring no administrator action after initial configuration.

Fifth, the self-hosted architecture ensures that all tenant personal data and financial records remain under the operator's direct control, satisfying data residency requirements without dependency on third-party cloud infrastructure.

Taken together, these contributions demonstrate that a purposefully scoped, full-stack web application built on a Node.js and MySQL foundation can address the specific administrative failures of the Vietnamese boarding house market more effectively than general-purpose tools or enterprise platforms scaled for different markets and user profiles.

7.2 Future Work

Three concrete development directions are proposed for the next phase of HostelHub, each addressing a specific limitation identified in section 6.4.

7.2.1 Native Mobile Application

The most impactful near-term extension is a native mobile application targeting Android and iOS. The cross-platform frameworks React Native and Flutter both support building a single codebase deployable to both platforms. A mobile client would consume HostelHub's existing REST API without requiring any changes to the backend. The primary capabilities to expose in the mobile interface are tenant invoice viewing, VNPay payment initiation, incident submission, and push notification receipt for payment reminders. Push notifications delivered through Firebase Cloud Messaging would replace the email-only reminder channel with a significantly higher delivery and open rate, addressing the limitation described in section 6.4.2. The administrator interface on mobile would focus on the high-frequency daily tasks of utility entry and incident status updates, while the full management dashboard would remain optimized for desktop browsers.

7.2.2 Multi-Channel Notification Integration via Zalo ZNS and SMS

HostelHub's current notification delivery relies exclusively on Gmail SMTP. Integrating Zalo ZNS, the official business notification API of the Zalo messaging platform, would deliver payment reminders and invoice notifications directly to tenants within the application they use most frequently in Vietnam. Zalo ZNS supports structured message templates with embedded payment links, making it a direct replacement for the current HTML email format. As a complementary channel for tenants who have not registered a Zalo account, integration with a domestic SMS gateway such as ESMS or SpeedSMS would provide universal coverage. Both Zalo ZNS and the SMS gateways expose REST APIs that Nodemailer's current role in the codebase could be supplemented by without requiring changes to the scheduling or notification database logic.

7.2.3 Predictive Analytics Using Machine Learning for Utility Forecasting and Revenue Modeling

The Utilities table accumulates monthly electricity and water consumption readings for every room over time. As this historical dataset grows, it provides sufficient signal for a time-series forecasting model to predict future consumption based on seasonal patterns and per-room usage trends. Anomalous months identified by the current 150% threshold rule could serve as labeled training data for a supervised classification model that learns more nuanced indicators of meter errors versus genuine consumption spikes. On the revenue side, the Invoices and Payments tables provide a structured dataset for occupancy rate forecasting and overdue probability scoring per tenant, which would allow the administrator dashboard to surface actionable risk signals rather than simply reporting current state. These capabilities could be delivered as a Python-based microservice consuming the existing MySQL database through a read-only connection, exposing prediction endpoints to the Node.js API layer without disrupting the core application architecture.

8. REFERENCES

1. AppFolio (2023) *AppFolio Property Manager: Cloud-based property management software* [Online]. Available at: <https://www.appfolio.com> (Accessed: 20 July 2026).
2. Bhimani, A. (2003) *Management accounting in the digital economy*. Oxford: Oxford University Press.
3. Buildium (2023) *Buildium property management software* [Online]. Available at: <https://www.buildium.com> (Accessed: 20 July 2026).
4. Chodorow, K. (2013) *MongoDB: The definitive guide*. 2nd edn. Sebastopol: O'Reilly Media.
5. Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, 13(6), pp. 377–387.
6. Collier, P.M. (2015) *Accounting for managers: Interpreting accounting information for decision making*. 5th edn. Chichester: Wiley.
7. Fielding, R.T. (2000) *Architectural styles and the design of network-based software architectures*. PhD thesis. University of California, Irvine.
8. General Statistics Office of Vietnam (2023) *Report on the results of the 2019 Vietnam housing and population census* [Online]. Available at: <https://www.gso.gov.vn> (Accessed: 15 July 2026).
9. Gray, J. and Reuter, A. (1992) *Transaction processing: Concepts and techniques*. San Francisco: Morgan Kaufmann.
10. Haerder, T. and Reuter, A. (1983) 'Principles of transaction-oriented database recovery', *ACM Computing Surveys*, 15(4), pp. 287–317.
11. Holovaty, A. and Kaplan-Moss, J. (2009) *The definitive guide to Django: Web development done right*. 2nd edn. New York: Apress.
12. IEEE (1998) *IEEE Std 830-1998: IEEE recommended practice for software requirements specifications*. New York: Institute of Electrical and Electronics Engineers.
13. Jones, M., Bradley, J. and Sakimura, N. (2015) *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. Available at: <https://datatracker.ietf.org/doc/html/rfc7519> (Accessed: 18 July 2026).
14. MISA (2023) *MISA AMIS: Accounting and business management software* [Online]. Available at: <https://amis.misa.vn> (Accessed: 20 July 2026).
15. National Assembly of Vietnam (2018) *Law No. 24/2018/QH14: Law on cybersecurity*. Hanoi: National Assembly of the Socialist Republic of Vietnam.
16. National Institute of Standards and Technology (2008) *FIPS PUB 198-1: The keyed-hash message authentication code (HMAC)*. Gaithersburg: U.S. Department of Commerce.
17. Node.js Foundation (2023) *About Node.js* [Online]. Available at: <https://nodejs.org/en/about> (Accessed: 15 July 2026).

18. Oracle Corporation (2023) *MySQL 8.0 reference manual* [Online]. Available at: <https://dev.mysql.com/doc/refman/8.0/en> (Accessed: 16 July 2026).
19. OWASP (2021) *OWASP top ten 2021* [Online]. Available at: <https://owasp.org/www-project-top-ten> (Accessed: 22 July 2026).
20. Provos, N. and Mazières, D. (1999) 'A future-adaptable password scheme', *Proceedings of the USENIX Annual Technical Conference*, Monterey, 6–11 June, pp. 81–91.
21. Richardson, L. and Ruby, S. (2007) *RESTful web services*. Sebastopol: O'Reilly Media.
22. Tilkov, S. and Vinoski, S. (2010) 'Node.js: Using JavaScript to build high-performance network programs', *IEEE Internet Computing*, 14(6), pp. 80–83.
23. Zalo (2023) *Zalo notification service (ZNS): Official business messaging API* [Online]. Available at: <https://developers.zalo.me/docs/zns> (Accessed: 20 July 2026).

APPENDIX: CORE SOURCE CODE IMPLEMENTATION

This appendix presents two complete source code implementations that represent the most technically significant automated and security-critical components of the HostelHub system. Each implementation is drawn directly from the production codebase and is annotated with inline comments to clarify the rationale behind each design decision.

Appendix A: Automated Cron Job Scheduler for Payment Reminders

A.1 Source Code

```
1 // server.js - Automated monthly payment reminder scheduler
2 // Dependencies: node-cron, nodemailer (transporter), mysql2 (pool)
3
4 const cron    = require('node-cron');
5 const path    = require('path');
6 const { exec } = require('child_process');
7 const fs      = require('fs');
8
9 /**
10  * Constructs the HTML body of a payment reminder email.
11  * Keeps email generation logic separate from scheduling logic
12  * to allow independent unit testing of the template.
13  */
14 function buildReminderHtml(invoice, paymentLink, formattedAmount) {
15     return `
16         <div style="font-family: Arial, sans-serif; max-width: 600px;
17 margin: auto;">
18             <h2 style="color: #e53935;">Payment Reminder</h2>
19             <p>Dear <strong>${invoice.name}</strong>,</p>
20             <p>
21                 This is a reminder that your invoice for
22                 <strong>Room ${invoice.room_number}</strong>
23                 covering the billing period
24
25 <strong>${invoice.billing_month}/${invoice.billing_year}</strong>
26                 remains unpaid.
27             </p>
28             <p>Amount due: <strong>${formattedAmount} VND</strong></p>
29             <a href="${paymentLink}"
30                 style="display:inline-block; padding:12px 24px;
31                     background:#1976d2; color:#fff;
32                     text-decoration:none; border-radius:4px;">
33                 Pay Now via VNPay
34             </a>
35             <p style="margin-top:24px; font-size:12px; color:#757575;">
36                 HostelHub Automated Billing System
37             </p>

```

```

38     </div>
39     `;
40 }
41
42 /**
43  * SCHEDULED TASK 1: Monthly Payment Reminder
44  * Cron expression '0 8 5 * *' executes at 08:00 on the 5th day of every
45 month.
46  * The task scans all unpaid invoices, dispatches a reminder email to
47 each
48  * associated tenant, and inserts an in-application notification record.
49  */
50 cron.schedule('0 8 5 * *', async () => {
51     console.log(`[CRON] Payment reminder task started at ${new
52 Date().toISOString()}`);
53
54     try {
55         // JOIN across Invoices, Rooms, and Users to retrieve all unpaid
56 invoices
57         // with the corresponding tenant contact details in a single
58 query,
59         // minimising the number of round-trips to the database server.
60         const [unpaidInvoices] = await pool.query(`
61             SELECT
62                 i.id            AS invoice_id,
63                 i.total_amount,
64                 i.billing_month,
65                 i.billing_year,
66                 u.id            AS user_id,
67                 u.name,
68                 u.email,
69                 r.room_number
70             FROM   Invoices i
71             JOIN   Rooms    r ON i.room_id   = r.id
72             JOIN   Users    u ON r.tenant_id = u.id
73             WHERE  i.status = 'unpaid'
74         `);
75
76         if (unpaidInvoices.length === 0) {
77             console.log(`[CRON] No unpaid invoices found. Task
78 complete.`);
79             return;
80         }
81
82         console.log(`[CRON] Processing ${unpaidInvoices.length} unpaid
83 invoice(s)...`);
84
85         for (const invoice of unpaidInvoices) {
86             // Construct a pre-signed payment URL that the tenant can
87 follow
88             // directly from the email without needing to log into the
89 portal first.

```

```

90         const paymentLink =
91             `${process.env.BASE_URL}/api/payment/create_payment_url`
92 +
93
94 `?invoice_id=${invoice.invoice_id}&amount=${invoice.total_amount}`;
95
96         const formattedAmount =
97             Number(invoice.total_amount).toLocaleString('vi-VN');
98
99         const mailOptions = {
100             from: `HostelHub` <${process.env.EMAIL_USER}>,
101             to: invoice.email,
102             subject: `[HostelHub] Payment reminder - ` +
103
104 `${invoice.billing_month}/${invoice.billing_year}`,
105             html: buildReminderHtml(invoice, paymentLink,
106 formattedAmount)
107         };
108
109         // Send reminder email via Gmail SMTP relay configured in
110 transporter
111         await transporter.sendMail(mailOptions);
112         console.log(`[CRON] Email sent to ${invoice.email} for
113 invoice #${invoice.invoice_id}`);
114
115         // Insert an in-application notification so the tenant sees
116 the
117         // reminder in the portal even if the email is not opened.
118         await pool.query(
119             `INSERT INTO Notifications (user_id, title, message)
120 VALUES (?, ?, ?)`,
121             [
122                 invoice.user_id,
123                 `Payment Reminder:
124 ${invoice.billing_month}/${invoice.billing_year}`,
125                 `Your invoice for Room ${invoice.room_number} ` +
126                 `totalling ${formattedAmount} VND is still unpaid. `
127 +
128                 `Please settle before the end of this month.`
129             ]
130         );
131     }
132
133     console.log('[CRON] Payment reminder task completed
134 successfully.');
```

```

135
136     } catch (err) {
137         // Log the full error stack to the server console for diagnosis.
138         // The task does not rethrow; a failure in one billing cycle
139         // must not crash the server process or prevent subsequent
140 cycles.
141

```

```

142         console.error('[CRON] Payment reminder task failed:',
143 err.message);
144     }
145 });
146
147
148 /**
149  * SCHEDULED TASK 2: Nightly Database Backup
150  * Cron expression '0 2 * * *' executes at 02:00 every night.
151  * Invokes the mysqldump binary to produce a full SQL dump of hostel_db,
152  * stored in the local backups directory with an ISO-formatted timestamp.
153  */
154 cron.schedule('0 2 * * *', () => {
155     const BACKUP_DIR = path.join(__dirname, 'backups');
156     const MYSQL_DIR = process.env.MYSQL_BIN_DIR ||
157 'C:\\xampp\\mysql\\bin';
158
159     // Ensure the backup directory exists before writing to it
160     if (!fs.existsSync(BACKUP_DIR)) {
161         fs.mkdirSync(BACKUP_DIR, { recursive: true });
162     }
163
164     // Replace colons and dots in the ISO string to produce a valid
165     filename
166     // across Windows and Unix filesystems.
167     const timestamp = new Date().toISOString().replace(/[:.]/g, '-')
168     '.slice(0, 19);
169     const filename = `auto_backup_${timestamp}.sql`;
170     const outputPath = path.join(BACKUP_DIR, filename);
171
172     // Shell command: mysqldump writes the complete schema and data to
173     stdout,
174     // which is redirected to the output file path.
175     const cmd =
176         `${MYSQL_DIR}\\mysqldump` +
177         ` -h ${process.env.DB_HOST}` +
178         ` -u ${process.env.DB_USER}` +
179         ` ${process.env.DB_NAME}` +
180         ` > "${outputPath}"`;
181
182     exec(cmd, err => {
183         if (err) {
184             console.error('[CRON] Nightly backup failed:', err.message);
185         } else {
186             console.log(`[CRON] Nightly backup created: ${filename}`);
187         }
188     });
189 });

```

A.2 Technical Explanation

The payment reminder scheduler is registered as a persistent background task within the Node.js process using the node-cron library, which implements the standard five-field cron expression syntax. The expression '0 8 5 * *' instructs the scheduler to fire at precisely 08:00 on the fifth calendar day of every month. The task body is declared as an async function, and the entire database interaction and email dispatch sequence is enclosed within a try-catch block. This is a deliberate resource safety decision: the catch handler logs the error message without rethrowing it, ensuring that an SMTP authentication failure or a transient database connection error in one monthly execution cycle does not crash the Node.js process or suppress the task from executing in subsequent months.

The database query within the task uses a three-table JOIN across Invoices, Rooms, and Users, retrieving all required tenant contact and billing data in a single round-trip rather than issuing individual queries per tenant. This design avoids the N+1 query pattern that would occur if the code first fetched a list of invoice IDs and then queried tenant details for each one separately. The subsequent iteration dispatches both an outbound SMTP email through Nodemailer and an in-application Notifications record for each unpaid invoice, providing two independent delivery channels that together satisfy functional requirements FR-08 and FR-17 as defined in Chapter 3.

Appendix B: VNPay Secure Payment Gateway Integration

B.1 Source Code

```
1 // routes/payment.routes.js - VNPay HMAC-SHA512 payment integration
2 // Dependencies: crypto (Node.js built-in), querystring (Node.js built-
3 in)
4 const crypto      = require('crypto');
5 const querystring = require('querystring');
6 const router      = require('express').Router();
7 const { pool }    = require('../config/db');
8
9 // VNPay configuration values loaded from the .env file.
10 // The vnp_HashSecret value must never be logged or exposed in responses.
11 const VNPAY_CONFIG = {
12   vnp_TmnCode: process.env.VNP_TMN_CODE,
13   vnp_HashSecret: process.env.VNP_HASH_SECRET,
14   vnp_Url: process.env.VNP_URL,
15   vnp_ReturnUrl: process.env.VNP_RETURN_URL,
16   vnp_IpnUrl: process.env.VNP_IPN_URL
17 };
18
19 /**
20  * Sorts an object's keys alphabetically after URL-encoding them,
```

```

21 * then URL-encodes each value with spaces represented as '+'.
22 *
23 * VNPay's signature specification requires that ALL parameters be sorted
24 * by URL-encoded key name before concatenation. Any deviation in sort
25 order
26 * produces a mismatched digest and causes the payment or IPN to be
27 rejected.
28 */
29 function sortObject(obj) {
30     const sorted = {};
31     const encodedKeys = Object.keys(obj)
32         .map(k => encodeURIComponent(k))
33         .sort();
34
35     for (const encodedKey of encodedKeys) {
36         const originalKey = decodeURIComponent(encodedKey);
37         sorted[encodedKey] = encodeURIComponent(obj[originalKey])
38             .replace(/%20/g, '+');
39     }
40     return sorted;
41 }
42
43 /**
44 * POST /api/payment/create_url
45 * Generates a signed VNPay payment URL for a specific invoice.
46 * The client redirects the user's browser to this URL to complete
47 payment
48 * on VNPay's hosted interface.
49 */
50 router.post('/create_url', async (req, res) => {
51     const { invoice_id, amount } = req.body;
52
53     if (!invoice_id || !amount) {
54         return res.status(400).json({ success: false, error: 'invoice_id
55 and amount are required.' });
56     }
57
58     // Retrieve the client's IP address for inclusion in the signed
59 parameter set.
60     // VNPay uses this for fraud detection and includes it in the
61 signature scope.
62     const ipAddr =
63         req.headers['x-forwarded-for'] ||
64         req.connection.remoteAddress ||
65         req.socket.remoteAddress ||
66         '127.0.0.1';
67
68     // Format the current timestamp as YYYYMMDDHHmmss in ICT (UTC+7).
69     // VNPay rejects requests whose vnp_CreateDate differs from server
70 time by
71 // more than fifteen minutes, providing replay attack protection.
72     const now = new Date();

```

```

73     const pad = n => String(n).padStart(2, '0');
74     const createDate =
75         `${now.getFullYear()}` +
76         `${pad(now.getMonth() + 1)}` +
77         `${pad(now.getDate())}` +
78         `${pad(now.getHours())}` +
79         `${pad(now.getMinutes())}` +
80         `${pad(now.getSeconds())}`;
81
82     // The transaction reference combines invoice_id with a millisecond
83 timestamp
84     // to guarantee uniqueness even if the same invoice is retried within
85 one second.
86     const txnRef = `${invoice_id}_${Date.now()}`;
87
88     let vnp_Params = {
89         vnp_Version: '2.1.0',
90         vnp_Command: 'pay',
91         vnp_TmnCode: VNPAY_CONFIG.vnp_TmnCode,
92         vnp_Locale: 'vn',
93         vnp_CurrCode: 'VND',
94         vnp_TxnRef: txnRef,
95         vnp_OrderInfo: `Pay invoice ${invoice_id}`,
96         vnp_OrderType: 'billpayment',
97         // VNPay amounts are expressed in VND multiplied by 100.
98         vnp_Amount: Math.round(parseFloat(amount) * 100),
99         vnp_ReturnUrl: VNPAY_CONFIG.vnp_ReturnUrl,
100        vnp_IpAddr: ipAddr,
101        vnp_CreateDate: createDate
102    };
103
104    // Step 1: Sort all parameters alphabetically by URL-encoded key.
105    vnp_Params = sortObject(vnp_Params);
106
107    // Step 2: Serialize to a URL-encoded query string without additional
108 encoding.
109    const signData = querystring.stringify(vnp_Params, { encode: false
110 });
111
112    // Step 3: Compute HMAC-SHA512 digest using the merchant secret key.
113    const hmac = crypto.createHmac('sha512',
114 VNPAY_CONFIG.vnp_HashSecret);
115    const signed = hmac.update(Buffer.from(signData, 'utf-
116 8')).digest('hex');
117
118    // Step 4: Append the computed digest as vnp_SecureHash.
119    // This parameter is NOT included in the signed string itself.
120    vnp_Params['vnp_SecureHash'] = signed;
121
122    const paymentUrl =
123        VNPAY_CONFIG.vnp_Url + '?' +
124        querystring.stringify(vnp_Params, { encode: false });

```

```

125
126     return res.status(200).json({ success: true, paymentUrl });
127 });
128
129 /**
130  * GET /api/payment/vnpay_ipn
131  * Server-to-server Instant Payment Notification handler.
132  * VNPay calls this endpoint after every transaction attempt.
133  * The handler MUST always respond with HTTP 200; the outcome is
134  * communicated through the RspCode field in the JSON body.
135  * A non-200 HTTP response causes VNPay to retry the IPN indefinitely.
136  */
137 router.get('/vnpay_ipn', async (req, res) => {
138     // Capture all incoming query parameters as a mutable object.
139     let vnp_Params = { ...req.query };
140
141     // Extract the submitted signature before removing it from the
142 parameter set.
143     // The signature must not be part of the string that is re-signed.
144     const receivedHash = vnp_Params['vnp_SecureHash'];
145
146     delete vnp_Params['vnp_SecureHash'];
147     delete vnp_Params['vnp_SecureHashType'];
148
149     // Re-derive the expected signature using the identical sort-and-
150 serialize
151     // procedure applied during URL construction.
152     vnp_Params = sortObject(vnp_Params);
153     const signData = querystring.stringify(vnp_Params, { encode:
154 false });
155     const hmac = crypto.createHmac('sha512',
156 VNPAY_CONFIG.vnp_HashSecret);
157     const computedHash = hmac.update(Buffer.from(signData, 'utf-
158 8')).digest('hex');
159
160     // Signature mismatch: the callback is rejected without any database
161 change.
162     // RspCode '97' instructs VNPay that the checksum failed.
163     if (receivedHash !== computedHash) {
164         console.warn('[IPN] Signature mismatch - callback rejected.');
```

```

165         return res.status(200).json({ RspCode: '97', Message: 'Checksum
166 failed' });
167     }
168
169     const responseCode = vnp_Params['vnp_ResponseCode'];
170     const txnRef = vnp_Params['vnp_TxnRef'];
171     const invoiceId = txnRef.split('_')[0];
172     const paidAmount = parseInt(vnp_Params['vnp_Amount'], 10) / 100;
173
174     // Only process transactions confirmed as successful by VNPay.
175     if (responseCode === '00') {
176

```

```

177         // Guard condition: UPDATE only affects rows with status =
178 'unpaid'.
179         // If VNPay retransmits the IPN after a network timeout, this
180 condition
181         // prevents the invoice from being updated twice and prevents a
182 duplicate
183         // Payment record from being inserted.
184         const [updateResult] = await pool.query(
185             `UPDATE Invoices SET status = 'paid' WHERE id = ? AND status
186 = 'unpaid'`,
187             [invoiceId]
188         );

189         if (updateResult.affectedRows > 0) {
190             // Record the transaction details for audit and
191             reconciliation.
192             await pool.query(
193                 `INSERT INTO Payments (invoice_id, vnpay_tran_id, amount)
194                 VALUES (?, ?, ?)`,
195                 [invoiceId, vnp_Params['vnp_TransactionNo'], paidAmount]
196             );

197             console.log(`[IPN] Invoice #${invoiceId} marked as paid. ` +
198                 `VNPay TxnNo:
199 ${vnp_Params['vnp_TransactionNo']}`);
200         } else {
201             // affectedRows = 0 indicates the invoice was already paid.
202             // Respond with success to suppress further VNPay retries.
203             console.log(`[IPN] Invoice #${invoiceId} already settled.
204 Duplicate IPN ignored.`);
205         }
206     }

207     // Respond with RspCode '00' in all non-signature-failure cases to
208     // acknowledge receipt and prevent VNPay from retrying the
209     notification.
210     return res.status(200).json({ RspCode: '00', Message: 'Confirm
211 Success' });
212 });

213 module.exports = router;

```

B.2 Technical Explanation

The payment URL construction process implements the exact signature protocol specified by VNPay's API documentation and the cryptographic standard defined in NIST FIPS PUB 198-1 (National Institute of Standards and Technology, 2008). The sortObject utility function is the most critical element of the integration: VNPay's server re-derives the expected signature from the same parameter set using an identical alphabetical sort, and any discrepancy in the ordering

of a single key produces a completely different HMAC-SHA512 digest, causing the payment request to be rejected before the tenant reaches the payment interface. The function URL-encodes each key before sorting to ensure that keys containing special characters are sorted by their encoded representation, which is the ordering convention VNPay's signature algorithm applies. The merchant secret key is accessed exclusively from the server-side environment configuration and is never included in any response body, URL parameter, or log output, satisfying the keyed-hash confidentiality requirement of HMAC as described in NIST FIPS PUB 198-1 (National Institute of Standards and Technology, 2008).

The IPN callback handler implements two independent layers of protection against fraudulent or malformed payment confirmations. The first layer is the HMAC-SHA512 signature verification, which rejects any callback whose computed digest does not match the submitted `vnp_SecureHash` value, regardless of the transaction result code it carries. The second layer is the idempotency guard embedded in the UPDATE statement, specifically the condition `AND status = 'unpaid'`, which ensures that a genuine VNPay retry caused by a network timeout does not insert a duplicate Payment record or trigger secondary business logic such as notification dispatch. The handler always responds with HTTP 200, because VNPay's integration specification interprets any non-200 HTTP status code as a server error and retransmits the IPN callback repeatedly until a 200 response is received. The outcome of the verification and database operation is communicated exclusively through the `RspCode` field in the JSON response body, which VNPay reads to determine whether to mark the transaction as acknowledged on its own ledger.